

SECTION 8 INSTRUCTION EXECUTION TIMING

This section gives the instruction execution times for the MC68881/MC68882 (FPCP) in terms of external clock cycles. This section provides the user with some reasonably accurate execution timing guidelines, but not exact timings for every possible circumstance. This approach is used since the exact execution time for an instruction is highly dependent on such things as external data formats, input operand values, operand type combinations, and timing relationships with respect to the main processor. The timing numbers presented in the following tables allow the assembly language programmer or compiler writer to predict worst-case timings needed to evaluate the performance of the FPCP or to optimize code for concurrent execution. Also, the effect that various data formats and operand values have on execution times can be observed to allow optimizing data structures for the highest performance for a given application. Finally, the timings for exception processing, context switching, and interrupt processing are included so that designers of multitasking or real-time systems can predict such things as task switch overhead and maximum interrupt latency due to floating-point operations.

When binary real data formats are used and no register conflicts occur, the MC68882 performs the operand transfer and data conversion for most general type instructions concurrently with calculations for preceding instructions. This section includes timing information that shows the amount of instruction overlap this concurrency provides.

8.1. FACTORS AFFECTING EXECUTION TIMES

When investigating instruction execution timing for the FPCP, it is assumed that the following information is required in order to make informed engineering trade-offs:

- Best case instruction execution timings, for determining whether or not an FPCP-based system can meet certain data processing performance criteria.
- Worst-case instruction timings and how they affect execution concurrency, to allow programs and compilers to be optimized to take maximum advantage of overlap under any timing circumstances.
- Guidelines to indicate how various programming practices can be utilized to improve upon the worst-case execution times, and thus allow performance to approach, as closely as possible, the best case execution times for a given task.
- The effects that an FPCP might have on system-related timings such as context switch overhead time in multitasking systems, or interrupt latency time in a real-time system.

First of all, when defining the performance of any machine that can operate in an asynchronous manner, or where data dependencies affect execution times, a set of assumptions must be made in order to provide a measurable environment. In this manual, instruction execution times are shown in clock cycles to remove clock frequency dependencies, and the following assumptions apply to define the context of the times shown.

- The main processor is an MC68020, acting as the host to the FPCP, and the two devices use the same clock input.



When the main processor initiates a command to the FPCP, any previous floating-point instruction has been completed and the FPCP is in the idle state.

- All operands in memory, as well as the system stack, are long word aligned.
- A 32-bit data bus is used for communications between the MC68020 and both the FPCP and the system memory.
- All memory accesses occur with no wait states (i.e., three clock cycle reads and writes).
- All coprocessor accesses, except those to the response and save CIRs, occur with no wait states. Accesses to the response and save CIRs require two wait cycles (five clock reads).

Note that the clock signal relationship between the MC68020/MC68030 (MPU) and the FPCP assumed for these discussions is not a system requirement, but merely a simplification that allows easy measurement of instruction times. However, the ratio of MPU clock frequency to FPCP clock frequency can be any reasonable value. In general, the clock frequency of the FPCP affects absolute instruction timing more than that of the MPU, since floating-point operations are usually computation intensive. However, the clock frequency relationship of the MPU and FPCP can affect the execution time of an instruction due to the time needed to transfer operands of various sizes and due to actual activity of the two devices. The magnitude of the dependency of execution times on the clock frequency of the MPU varies with instruction types, since some instructions spend a relatively small amount of their overall execution time in communication with the main processor; whereas, other instructions spend almost all of their execution time in communication with the main processor.

8

With this set of assumptions as a starting point, several factors must be defined that contribute to the overall execution time for a given instruction. Some of these factors are common to all instructions, while others are only applicable to certain instructions or data types. Particularly, the execution times for the conditional and system control instructions are not widely variable, but the execution time for an arithmetic or data movement instruction is heavily affected by data values and exception checking. In order to better understand how these factors are combined to calculate the execution time for an arithmetic or move-to-floating-point register instruction, it is helpful to divide coprocessor instruction execution into the following steps:

1. Receive the command word from the host processor, decode it, and return the first service request primitive.
2. Receive the main processor program counter, if required.
3. Receive an external operand, if required.
4. Convert the operand to the internal extended format.
5. Perform the algorithm specified by the command word on the operand(s).
6. Round the result to the correct precision, check the result of the computation for conditions such as overflow, then store the result into a floating-point data register.

The first three of these steps require approximately the same amount of time for any instruction, but the last three steps can require widely varying amounts of time even when comparing the execution time for a given instruction with different data inputs. For purposes of this discussion, the first three steps are referred to as the instruction start-up phase, the fourth step as the conversion phase, the fifth step as the calculation phase, and

8.1.1 Instruction Start-Up Phase

The factor that affects execution time most heavily during this phase of an instruction is the location and format of an external operand. The three possible locations for an input operand are:

1. In a floating-point data register,
2. In a main processor data register, and
3. In external memory.

If an operand resides in a floating-point data register before an instruction starts, no data movement operation is required to prepare it for the calculation phase, and thus, the start-up phase is very short. If an operand resides in a main processor data register, the FPCP uses the evaluate effective address and transfer data response primitive to request it from the MPU. In this case, the MPU does not generate any operand memory cycles, and the operand is transferred to the FPCP with a single bus cycle. The FPCP then converts the signed integer or single precision floating-point number to extended precision and proceeds to the calculate phase.

For the third operand location case, execution time can vary widely due to two separate mechanisms, the addressing mode and alignment of the operand in memory, and the data format and value of the operand. The addressing mode used to locate an operand affects execution time in a straightforward manner due to the fixed nature of effective address calculations by the MPU. For example, if the addressing mode used is address register indirect, (An), then no instruction prefetch words and one long-word indirect address fetch may be required to calculate the final address of the operand. Then, once the operand is located, up to three long word fetches may be required to transfer the operand to the FPCP. The execution times for these operations are quite predictable (i.e., there are no data dependencies involved), although they are affected by instruction stream alignment, MPU cache hits, memory access times, memory width, and operand alignment. As mentioned earlier, certain assumptions are made with regard to these factors (for the purposes of this discussion) so that the tables in this section may be simplified. In order to include the effects of these factors, refer to the MC68020 user's manual or the MC68030 user's manual for more information regarding bus operation.

The second mechanism that can affect execution times for operands in memory is the data format. For the integer and binary floating-point formats, the execution times for conversions required to prepare the operand for the calculation are relatively free from data dependencies. However, for the packed decimal floating-point format, execution times can vary significantly due to the value of the input operand.

8.1.2 Calculation Phase

This is the most volatile portion of an instruction with respect to execution times. The main factor that affects the calculation time is the operation to be performed (e.g., a sine operation requires far more time than an add operation), but for a given operation, the execution time is data dependent. For the monadic operations, the data dependency is limited to the type and value of the input operand; for the dyadic operations, the combination of the

is and values of the two operands can also affect execution time (in this context, data refers to the FPCP-extended precision representation of one of the five IEEE data types: normalized, denormalized, zero, infinity, and not-a-number). Because execution times vary due to data values and type combinations, Tables 8-14 and 8-15 indicate the execution time for each arithmetic operation with typical arguments, along with timing values for special case operand types such as zero, infinity, etc.

8.1.3 Round/Store Result Phase

The execution time for this phase of an instruction is dependent on the mode of operation that is programmed into the floating-point control register, as well as the value of the result. For example, if the rounding precision is programmed to be extended, execution is faster than if it is single. Also, if the result of a calculation overflows or underflows the destination precision, then more time is required to handle that exception. In the following paragraphs, the overall execution times for the arithmetic operations assume the best case round/store phase time. Table 8-16 lists the values used to calculate execution times for various rounding precisions and exception handling operations.

8.2 CONCURRENT INSTRUCTION EXECUTION

8

An important factor that should be considered when optimizing MPU and FPCP programs is the amount of concurrent execution time that an instruction allows. It is also an important consideration in calculating overall execution time.

Concurrency between MPU and the FPCP applies when the main processor executes MPU instructions while the coprocessor completes execution of a floating-point instruction. The MC68882 can execute two floating-point instructions concurrently, providing additional concurrency not available in the MC68881.

Overlap time between instructions determines the degree of concurrency that is possible. Overlap time is derived from the combination of the tail of an instruction with the head of the next instruction, where tail and head are portions of the total execution time of an instruction. The tail is the portion of the total execution time during which another instruction can be executed. The head is the portion of the total execution time that can be performed while another instruction is completing. The overlap time of two consecutive instructions is either the tail of the first instruction or the head of the second, whichever is less.

The tail of a floating-point instruction is the time during which the MPU can execute a subsequent instruction. It consists of the time during which the processor releases the MPU to allow a subsequent instruction to begin. During this period, the coprocessor is still performing the calculations necessary to complete the current instruction.

In the case of the MC68881, overlap occurs only when the subsequent instruction is an MPU instruction. Table 8-25 is used to calculate the portion of the MC68881 instruction that can overlap with an MPU instruction. If the subsequent instruction is a floating-point instruction, the MPU is requested to wait to execute it until the coprocessor finishes the current instruction. That is, the head portion of the execution time for a floating-point instruction executing in an MC68881 is zero.



MC68882, however, can obtain the operand of a subsequent floating-point instruction convert the operand to internal format during the tail of the previous instruction. This portion of the instruction is defined as the head of the MC68882 instruction. It is the portion of the instruction that begins when the instruction is initiated by the MPU, and ends when the present coprocessor instruction can no longer operate under the tail of a previous instruction. The actual values of head and tail that apply to the MC68882 floating-point instructions are shown in Table 8-3. The tails of MC68882 floating-point instructions can execute concurrently with MPU instructions as well as with other MC68882 instructions. The head times for the MC68882 instructions indicate the degree of concurrency that is allowed.

Each floating-point instruction of the MC68882 is either fully concurrent, partially concurrent, or nonconcurrent. Instructions for which the head time equals the total execution time are fully concurrent. Those for which head and tail values are shown are partially concurrent. The instructions that have zero head values are noncurrent.

Concurrent execution of floating-point instructions in the MC68882 can significantly improve coprocessor performance. Refer to **5.1.2 Optimization of Code for the MC68882** for more information on the effects of concurrency on the performance of programs.

8.3 INTERRUPT LATENCY TIMES

In real-time systems, a very important factor pertaining to overall system performance is the response time required for a processor to handle an interrupt. In the M68000 Family of processors, interrupts are allowed to be asserted to the processor asynchronously, and they are handled on the next instruction boundary. While the average interrupt latency for the MPU is quite short, the maximum latency is often of critical importance since real-time interrupts cannot require servicing in less than the maximum interrupt latency. The maximum interrupt latency for the MPU is approximately 250 clock cycles (for the MOVEM.L ([d32,An],Xn,d32), D0-D7/A0-A7 instruction where the last data fetch is aborted with a bus error; refer to the MC68020 user's manual or the MC68030 user's manual for more detailed information), but some FPCP instructions may take two or three times that long to execute with typical operand types, combinations and values.

It may be unacceptable in a real-time system to have a worst-case interrupt latency time as large as 600 or more clock cycles (the length of some long floating-point instructions). Therefore, the FPCP allows interrupts to be processed in the middle of the execution of a floating-point instruction, whenever possible, to reduce the latency time. The FPCP does this in four ways:

1. By returning the null (CA=0, IA=1, PF=0) primitive when it enters the calculated phase of an instruction that allows concurrency. If the MPU is not in the trace mode, it is then free to fetch the next instruction and process any pending interrupts at the instruction boundary. If the MPU is in the trace mode, it waits for the FPCP to complete execution and return the null (CA=0, PF=1) primitive before continuing with the next instruction, but it services pending interrupts while it is waiting.
2. By returning the null (CA=1, IA=1) primitive when the main processor attempts to initiate a floating-point instruction while the FPCP is unable to begin another operation, thus allowing the MPU to service interrupts while waiting for the coprocessor to start execution of the new instruction.



By returning the null (CA=1, IA=1) primitive during internal conversions for non-concurrent instruction execution (e.g., FMOVE.<fmt>FPn,<ea> in the MC68881 or FMOVE.W FPn,<ea> in the MC68882) before returning service request primitives to complete the operation.

4. By returning the not-ready, come-again format code during internal operations required by the FSAVE instruction. The FPCP returns this format code in some cases (as described in **6.4 CONTEXT SWITCHING**) to enable it to store a smaller state frame, and the MPU can process interrupts while waiting for the save operation to begin.

For the first two cases, the MPU is allowed to process interrupts during the tail period defined in **8.2 CONCURRENT INSTRUCTION EXECUTION**. For the third case, the period during which the MPU can process interrupts is illustrated in Figure 8-1. The timing for the fourth case is similar to the third case, except that the periods labeled "Convert," "Round," and "Transfer" for the FPCP are not used for those purposes but instead for saving of the internal state.

MC68020/MC68030	INITIATE INSTRUCTION	WAIT INTERRUPTS ALLOWED			STORE
MC688881/MC68882	START-UP	CONVERT	ROUND	TRANSFER	

Figure 8-1. Nonconcurrent Instruction Execution, Interrupts Allowed

8

Basically, the maximum interrupt latency time for any FPCP instruction is equal to the worst-case execution time minus the interrupts allowed time, where both of these values are calculated using the tables in this section. For concurrent instructions, the execution time and allowed concurrency times are shown, and the interrupt latency is the difference between these two values. For nonconcurrent instructions, the amount of time during which interrupts are allowed is shown in the tables as the number of allowed overlap clock cycles, and the interrupt latency is approximately equal to the total execution time minus the allowed overlap time. However, as shown in Figure 8-1, there may be two separate time periods during which the MPU is not allowed to process interrupts. For some instructions, such as the FMOVE.P FPn,<ea> instruction, these two periods are approximately equal and make up a small fraction of the overall execution time for the operation. On the other hand, for the FRESTORE and FSAVE instructions, the time required to transfer a busy state frame is roughly equal to the overall execution time. In fact, the worst-case interrupt latency due to an FPCP instruction is for the FRESTORE instruction with a busy state frame.

8.4 COPROCESSOR INTERFACE OVERHEAD

For all of the instruction timings shown in the following tables, all coprocessor interface bus cycle timing and associated processing are included in the overall execution times. However, it is assumed that when the main processor begins execution of a floating-point instruction, the FPCP has completed execution of any previous instruction and is ready to begin a new instruction. (Note that the criteria for determining the readiness of the MC68881 is different from that of the MC68882. The MC68881 is ready to begin an instruction if the previous instruction is completed. The MC68882, however, is ready to begin an instruction if the instruction has completed executing in the CU and the CU has handed off the instruction to the APU.) Thus, the MPU is never required to wait while the FPCP completes



instruction. Also, it is assumed that when the MPU is waiting for the FPCP during a concurrent instruction, the main processor reads the response register at exactly the moment when the FPCP is prepared to return a service request primitive to complete or continue the instruction. If these conditions are not met, the actual instruction execution time can be shorter or longer than the values shown in the tables, due to synchronization of the two devices.

First, it must be noted that the FPCP does not begin execution of an instruction until the start of the read cycle from the response CIR in which the first primitive of the instruction dialog is returned to the main processor. If the MPU attempts to initiate a floating-point instruction before the previous one has completed and the coprocessor is ready, the FPCP queues the command word or conditional predicate and then instructs the MPU to wait (by encoding the null (CA=1, IA=1) primitive in the response CIR) until the previous instruction is completed and the FPCP is ready to begin the next instruction. When the previous instruction has completed execution, the FPCP does not begin execution of the queued instruction until the next read of the response CIR. The sequence of events for this situation is:

1. The FPCP allows concurrent instruction execution by returning the null (CA=0, IA=1, PF=0) primitive to the MPU.
2. The MPU encounters the next FPCP instruction and attempts to initiate execution by writing to the command or condition CIR. The MPU then starts a read from the response CIR to determine what further action should be taken.
3. The FPCP queues the instruction initiation request and changes the encoding of the response CIR to null (CA=1, IA=1), causing the MPU to wait.
4. The MPU continues to read the response CIR repeatedly until a new primitive is encoded or an interrupt becomes pending (if an interrupt occurs, the MPU resumes polling of the response CIR after the interrupt handler executes an RTE instruction).
5. The MC68881 APU becomes idle (by completing the previous instruction) and waits for the next read of the response CIR. In the MC68882, the CU hands off the instruction to the APU after the APU completes the previous instruction and waits for the next read of the response CIR.
6. The MPU reads the response CIR, which either results in the return of a take exception primitive (due to an exception during the previous instruction) or causes the FPCP to begin execution of the new instruction by returning the first primitive required for that operation.

The timing relationship of the main processor and the FPCP during this sequence can affect the overall execution time of the new instruction, due to synchronization between the two devices. Specifically, if the MPU begins a read of the response CIR exactly one clock cycle before the FPCP completes the execution of the previous instruction in the APU, the FPCP immediately begins execution of the new instruction by returning the first primitive of the new instruction dialog during that read cycle. This case is shown in Figure 8-2, which illustrates the best-case timing for coprocessor interface overhead: two clock cycles.

Figure 8-2 also illustrates the typical coprocessor interface overhead timing, which occurs when the MPU initiates a new instruction and the FPCP is in the idle state. For this case, there is no overlap with a previous instruction at the beginning of the instruction dialog, and the coprocessor interface overhead for the new instruction is 11 clock cycles. Also,

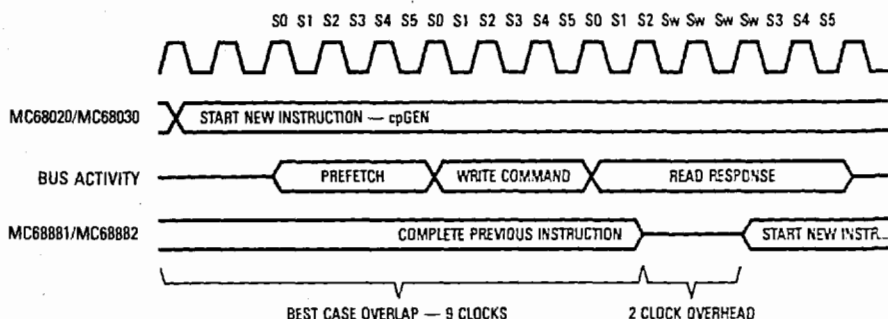


Figure 8-2. Best-Case Coprocessor Interface Overhead Timing

note that this example assumes that the instruction prefetch requested by the cpGEN start-up operation was not satisfied by the previous prefetch bus cycle, and it does not hit in the MPU on-chip instruction cache. Refer to **8.5.2 MC68881 Detail Timing Tables** for further discussion of the effects of instruction prefetching by the MPU.

If the read cycle to the response CIR occurs before the FPCP has completed execution of the previous instruction, the MPU processes the null (CA = 1, IA = 1) primitive that is returned (by checking for pending interrupts and re-reading the response CIR if there are none). This operation requires 10 MPU clock cycles, and thus there is a 10 clock cycle worst-case synchronization period that is part of the effective execution time for the new instruction. (The worst case occurs if the response CIR read cycle starts two clock cycles before the previous instruction in the APU is completed.) The worst-case timing is shown in Figure 8-3. The exact amount of synchronization time required is dependent on the system environment, such as the clock signal relationship between the MPU and the FPCP, and the context of an instruction sequence. In all of the following tables, the typical case of no overlapped execution is assumed; thus, a coprocessor interface overhead value of 11 clock cycles is included in the timing numbers. If an attempt is made to optimize an instruction sequence for overlapped execution, the coprocessor interface overhead may be reduced by as much as nine clock cycles. However, incorrect "optimization" may result in an 11 clock cycle overhead, which is no worse than the no overlap case previously described.

A similar overhead effect occurs for nonconcurrent instructions that allow interrupt processing by returning the not-ready format code (FSAVE instruction), or the null (CA = 1, IA = 1) primitive in the middle of instruction execution (i.e., the FSAVE and FMOVE Fp, <ea> instructions). In these cases, the FPCP completes as much of the instruction as possible while allowing interrupts, and then prepares to encode a valid format code or service request primitive in the save or response CIR during the next read by the MPU. The timing relationship between the start of the read cycle and the completion of internal operations by the FPCP is identical to the timing previously described for the instruction start-up phase. Thus, the same 10 clock cycle overhead factor might be added to the execution time for these instructions.

In the following tables, the assumptions stated earlier apply (i.e., the main processor is an MPU running on the same clock as the FPCP), and the coprocessor interface overhead for all operations other than instruction initiation is included based on those assumptions. If an instruction is executed under conditions other than those described, the execution time may be increased in increments of 10 clock cycles if necessary.

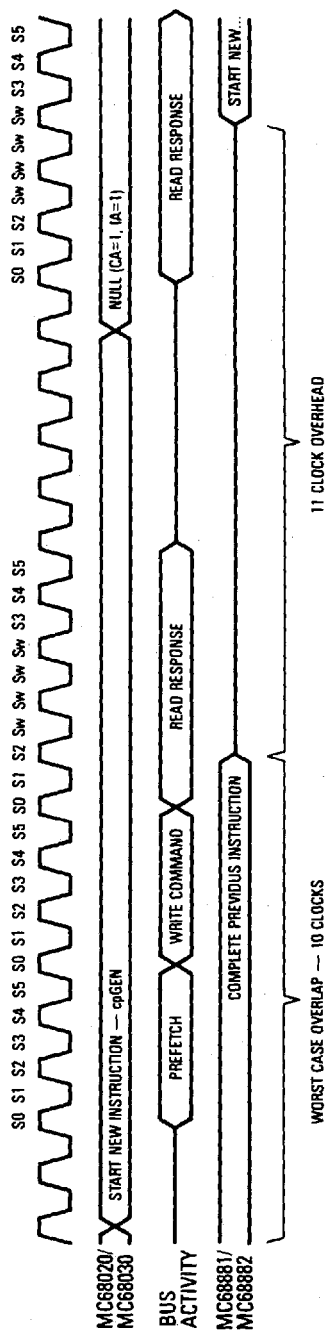


Figure 8-3. Worst-Case FPCP Interface Overhead Timing

following paragraphs, timing tables are presented that allow the calculation of best-case, typical, and worst-case execution times for any FPCP instruction. These tables are based on the assumptions previously stated and include the total execution time for each instruction. In other words, the numbers that are calculated using these tables indicate the time from the beginning of execution of the coprocessor instruction by the MPU (i.e., when the instruction has been prefetched and loaded into the instruction decode register) to completion of the instruction by the FPCP and/or MPU (i.e., when a read of the response CIR indicates a null (CA=0, PF=1), when conditional processing is completed, or when the last operand transfer to or from the FPCP has been completed).

Bus cycle activity is also indicated by the tables and includes all bus cycles generated by a particular operation. Note that instruction prefetch and operand write cycles requested by the execution of a given instruction may not actually be executed during the execution of the instruction, but are queued by the MPU bus interface unit for completion as soon as the bus is available. (Refer to the MC68020 and MC68030 user's manual for more information on bus cycle overlap.) When a floating-point operation is completed, a prefetch request has been generated by the MPU to replace each word of the instruction stream used by the instruction or to refill the instruction pipe in the case of a conditional branch, a trap instruction, or an exception.

The timing information shown in the following tables for some operations includes three numbers that depend on the context of the instruction (i.e., the alignment of the instruction stream, whether the MPU instruction cache is enabled, and whether the cache contains the instruction.)

8

1. The best-case value, where prefetches hit in the MPU on-chip cache and the instruction benefits from the maximum overlap, in the MPU pipeline, with other instructions. Due to the highly volatile nature of the instruction pipeline, this case is not easy to achieve intentionally but occurs occasionally.
2. The cache-only case, where prefetches hit in the MPU on-chip cache, but the instruction does not overlap with preceding or following instructions.
3. The worst-case, where prefetches do not hit in the MPU on-chip cache or the cache is disabled, and there is no instruction overlap. It is further assumed that the instruction is aligned so that a prefetch is executed before the MPU writes to the FPCP command CIR.

The execution time entries in most of the following tables contain seven numbers. The left-most number is the total execution time for the operation in clock cycles, followed by the number of clock cycles of the total execution time that is allowed to overlap with execution of other operations by the main processor. Then, in parenthesis, the bus cycle activity is included, which indicates the number of instruction prefetch, operand read, operand write, coprocessor read, and coprocessor write bus cycles that are generated by the execution of the instruction. An example of the format of an entry from the timing table is:

	xx/xx	(xx/xx/xx/xx/xx)
Total execution time	_____	_____
Overlap with subsequent MPU instructions	_____	_____
Number of prefetch bus cycles	_____	_____
Number of operand read bus cycles	_____	_____
Number of operand write bus cycles	_____	_____
Number of coprocessor read bus cycles	_____	_____
Number of coprocessor write bus cycles	_____	_____

total number of clocks required for the bus activity in each entry can be derived by plying the total number of bus cycles by three. (This does not account for the fact that reads from the response and save CIRs require five clocks rather than three, but the two clock-cycle discrepancy is usually negligible compared with the overall execution time for an instruction.) For some instructions, the number of coprocessor read cycles indicated by the tables may not reflect the actual number of read cycles that are executed during the dialog for the instruction. This is because only the first occurrence of a series of null (CA=1, IA=1) or null (CA=0, PF=0, IA=1) primitives is included in the tables. For example, the FPCP forces the main processor to wait during the conversion phase of the FMOVE FPN,<ea> instruction by using the null response primitive. The MPU may perform numerous response CIR read cycles during the time that it waits for the FPCP, but only the first of this series of read cycles is included in the timing table entry. Although this simplification may fail to indicate the true number of coprocessor read cycles executed by the MPU, it allows the tables to accurately indicate the minimum number of different response primitive and operand reads that must be executed by a main processor, regardless of its type or clock and bus speed.

The timing tables in the following paragraphs are divided into two major groups. First, several tables are presented that allow quick determination of the typical execution time for all instructions. These tables are comprehensive but assume typical operand inputs and operating conditions for simplicity. No more than two tables are used to determine the typical execution time for a given instruction. One table is used to determine the basic execution time for the selected instruction, and a second table (one of five listing the instruction groups) is used to determine the additional time required for the calculation of the effective address by the MPU, for those instructions that require an effective address calculation.

The second group of tables is used to calculate a more precise execution timing value for a specific instruction, addressing mode, and operand type combination than is available in the first group of tables. This group of tables is also useful for the calculation of execution times where the main processor is not a MPU, since the timing for each phase of instruction execution is included in a separate table. This allows timings that are only dependent on the FPCP to be calculated and added to the timing characteristic of the main processor.

8.5.1 Timing Tables for Typical Execution

This set of tables allows a quick determination of the typical execution time for any FPCP instruction when the MPU is used as the main processor. The first table presented is for effective address calculations performed by the MPU. Entries from this table are added to the entries in the other tables in this subsection, if necessary, to obtain the overall execution time for an operation. The assumptions that apply to the following tables are:

- The main processor is an MC68020 and operates on the same clock as the FPCP. Instruction prefetches do not hit in the MC68020 cache (or it is disabled), and the instruction is aligned so that a prefetch occurs before the command CIR is written by the MC68020.

NOTE

The timing numbers are derived assuming that the main processor is an MC68020. The MC68030 has a more optimized coprocessor interface and can benefit from the data cache hits. These improvements of the coprocessor interface are not used in determining typical operation. Actual operation when using the MC68030 always yields better values than the calculations derived from these tables.

A 32-bit memory interface is used, and memory accesses occur with zero wait states. All memory operands, as well as the stack pointers, are long word aligned.

- Accesses to the FPCP require three clock cycles, with the exception of read accesses to the response and save CIRs, which require five clock cycles.
- No instruction overlap is utilized so the coprocessor interface overhead is 11 clocks. This can be reduced to two clock cycles if optimized code sequences are used or may be 11 clock cycles if overlap is attempted and a synchronization delay is required.
- Typical operand conversion and calculation times are used (i.e., input operands are assumed to be normalized numbers in the legal range for a given function).
- No exception is enabled, no exception occurs, and the default rounding mode and precision of round-to-nearest, extended precision is used.

8.5.1.1 EFFECTIVE ADDRESS CALCULATIONS. For any instruction that requires an operand external to the FPCP, an evaluate effective address and transfer data response primitive is issued by the FPCP during the dialog for that instruction. The amount of time that is required for the MPU to calculate the effective address while processing this primitive for each addressing mode, excluding the transfer of the data to the FPCP, is shown in Table 8-1. The times shown in this table include all bus cycles required to perform the address calculation (such as instruction prefetches and memory indirect fetches).

For the FMOVEM instruction, Table 8-1 is also used to determine the time required for the MPU to perform an address calculation (implied by the transfer multiple coprocessor registers primitive). For the FScc, FRESTORE, and FSAVE instructions, the request to evaluate an effective address is implied by the F-line instruction word; therefore, no response primitive is issued by the FPCP to request the evaluation. The following table is used for these three instructions to adjust the basic instruction execution time to reflect the addressing mode that is used.

Note that Table 8-1 applies only to the MPU effective address calculation time for coprocessor instructions. The execution times included in this table are not the same as the calculate effective address times given in the MPU user's manuals for normal instruction execution.

8.5.1.2 ARITHMETIC OPERATIONS. Three tables provide the typical instruction execution time for each arithmetic instruction. This group of instructions includes the majority of the FPCP operations such as FADD, FSUB, etc. In addition to the instructions that perform arithmetic calculations as part of their function, the FCMP, FMOVE, and FTST instructions are also included since an implicit conversion is performed by those operations. For memory operands, the timing for the appropriate effective addressing mode must be added to the numbers in these tables to determine the overall instruction execution times. In order to simplify these tables, the overall execution times for the MC68881 are listed in Table 8-2, the overall execution times for the MC68882 are listed in Table 8-3, and the bus cycle activity numbers are listed in Table 8-4. In addition to the total execution times for the MC68882, Table 8-3 lists the head and tail values required for calculating concurrency.

Table 8-1. Effective Address Calculations

Addressing Modes	Best Case	Cache Case	Worst Case
Dn or An	0/0 (0/0/0/0/0)	0/0 (0/0/0/0/0)	0/0 (0/0/0/0/0)
{An}	0/0 (0/0/0/0/0)	2/0 (0/0/0/0/0)	2/0 (0/0/0/0/0)
{An} +	3/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)
- {An}	3/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)
{d ₁₆ ,An} or {d ₁₆ ,PC}	0/0 (0/0/0/0/0)	2/0 (0/0/0/0/0)	3/0 (1/0/0/0/0)
{xxx}.W	0/0 (0/0/0/0/0)	2/0 (0/0/0/0/0)	3/0 (1/0/0/0/0)
{xxx}.L	1/0 (0/0/0/0/0)	4/0 (0/0/0/0/0)	5/0 (1/0/0/0/0)
#<data>	0/0 (0/0/0/0/0)	0/0 (0/0/0/0/0)	0/0 (0/0/0/0/0)
{d ₈ ,An,Xn} or {d ₈ ,PC,Xn}	1/0 (0/0/0/0/0)	4/0 (0/0/0/0/0)	5/0 (1/0/0/0/0)
{d ₁₆ ,An,Xn} or {d ₁₆ ,PC,Xn}	3/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	7/0 (1/0/0/0/0)
{B}	3/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	7/0 (1/0/0/0/0)
{d ₁₆ ,B}	5/0 (0/0/0/0/0)	8/0 (0/0/0/0/0)	9/0 (1/0/0/0/0)
{d ₃₂ ,B}	11/0 (0/0/0/0/0)	14/0 (0/0/0/0/0)	16/0 (2/0/0/0/0)
{[B],l}	8/0 (0/1/0/0/0)	11/0 (0/1/0/0/0)	12/0 (1/1/0/0/0)
{[B],l,d ₁₆ }	8/0 (0/1/0/0/0)	11/0 (0/1/0/0/0)	12/0 (1/1/0/0/0)
{[B],l,d ₃₂ }	10/0 (0/1/0/0/0)	13/0 (0/1/0/0/0)	15/0 (2/1/0/0/0)
{[d ₁₆ ,B],l}	10/0 (0/1/0/0/0)	13/0 (0/1/0/0/0)	14/0 (1/1/0/0/0)
{[d ₁₆ ,B],l,d ₁₆ }	10/0 (0/1/0/0/0)	13/0 (0/1/0/0/0)	15/0 (2/1/0/0/0)
{[d ₁₆ ,B],l,d ₃₂ }	12/0 (0/1/0/0/0)	15/0 (0/1/0/0/0)	17/0 (2/1/0/0/0)
{[d ₃₂ ,B],l}	16/0 (0/1/0/0/0)	19/0 (0/1/0/0/0)	21/0 (2/1/0/0/0)
{[d ₃₂ ,B],l,d ₁₆ }	16/0 (0/1/0/0/0)	19/0 (0/1/0/0/0)	21/0 (2/1/0/0/0)
{[d ₃₂ ,B],l,d ₃₂ }	18/0 (0/1/0/0/0)	21/0 (0/1/0/0/0)	24/0 (3/1/0/0/0)

B = Base address; 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

l = Index; 0 or Xn.

NOTE: Xn cannot be in B and l at the same time. Scaling and size of Xn does not affect timing.

8.5.1.3 MC68882 CONCURRENT OPERATIONS. The MC68882 overall instruction timing table, Table 8-3, contains the H and T numbers, which are helpful in estimating the instruction execution overlap resulting from concurrent execution of floating-point instructions.

- **H — Head.** The effective address calculation should be added to the head to obtain the true head time. (This does not apply for FMOVE to memory if a register conflict occurs.)
- **T — Tail.** The period during which the MC68882 can begin another floating-point instruction.

The total execution time for a set of instructions is the sum of the overall execution times of the individual instructions in the set minus the total overlap time. This formula applies to both the MC68881 and the MC68882; for the MC68881, the overlap between floating-point instructions is zero.

Table 8-5 lists an example of the use of the timing tables to calculate the execution time for a sequence of instructions. The table compares the execution times, in clock cycles, of the individual instructions and the total execution time for the sequence using the MC68881

Table 8-2. MC68881 Overall Execution Times

Instruction	FPn to FPM	Memory Source or Destination Operand Format				
		Integer	Single	Double	Extended	Packed
FABS	35	62	54	60	58	872
FACOS	625	652	644	650	648	1462
FADD	51	80	72	78	76	888
FASIN	581	608	600	606	604	1418
FATAN	403	430	422	428	426	1240
FATANH	693	720	712	718	716	1530
FCMP	33	62	54	60	58	870
FCOS	391	418	410	416	414	1228
FCOSH	607	634	626	632	630	1444
FDIV	103	132	124	130	128	940
FETOX	497	524	516	522	520	1334
FETOXM1	545	572	564	570	568	1382
FGETEXP	45	72	64	70	68	882
FGETMAN	31	58	50	56	54	858
FINT	55	82	74	80	78	892
FINTRZ	55	82	74	80	78	892
FLOGN	525	552	544	550	548	1352
FLOGNP1	571	598	590	596	594	1408
FLOG10	581	608	600	606	604	1418
FLOG2	581	608	600	606	604	1418
FMOD	70	99	91	97	95	907
FMOVE to FPN	33	60	52	58	56	870
FMOVE to memory	—	100	80	86	72	2002
FMOVECR	29	—	—	—	—	—
FMUL	71	100	92	98	96	908
FNEG	35	62	54	60	58	872
FREM	100	129	121	127	125	937
FSCALE	41	70	62	68	66	878
FSGDIV	69	98	90	96	94	906
FSGLMUL	59	88	80	86	84	896
FSIN	391	418	410	416	414	1228
FSINCOS	451	478	470	476	474	1288
FSINH	687	714	706	712	710	1524
FSQRT	107	134	126	132	130	944
FSUB	51	80	72	78	76	888
FTAN	473	500	492	498	496	1310
FTANH	661	688	680	686	684	1498
FTENTOX	567	594	586	592	590	1404
FTST	33	60	52	58	56	870
FTWOTOX	567	594	586	592	590	1404

*Add the appropriate effective address calculation time.

**If the source or destination is an MPU data register, subtract five or two clock cycles, respectively.

***Assumes a static k factor is used if the destination data format is packed decimal. Add 14 clock cycles if a dynamic k factor is used.

****The source operand is from the constant ROM rather than a floating-point data register.

Table 8-3. MC68882 Overall Execution Times

Monadic	Memory Source or Destination Operand Format***																	
	FPn to FPm			Integer****			Single****			Double			Extended			Packed		
	H	T	Total	H	T	Total	H	T	Total	H	T	Total	H	T	Total	H	T	Total
FABS	17	17	38	21	28	68	30	20	51	36	20	57	42	20	63	13	811	893
FACOS	17	607	628	21	618	658	30	610	641	36	610	647	42	610	653	13	1401	1483
FADD	17	35	56	21	54	94	30	38	69	36	38	75	42	38	81	13	827	909
FASIN	17	563	584	21	574	614	30	566	597	36	566	603	42	566	609	13	1357	1439
FATAN	17	385	406	21	396	436	30	388	419	36	388	425	42	388	431	13	1179	1261
FATANH	17	675	696	21	686	726	30	678	709	36	678	715	42	678	721	13	1469	1551
FCMP	17	17	38	21	36	76	30	20	51	36	20	57	42	20	63	13	809	891
FCOS	17	373	394	21	384	424	30	376	407	36	376	413	42	376	419	13	1167	1249
FCOSH	17	589	610	21	600	640	30	592	623	36	592	629	42	592	635	13	1383	1465
FDIV	17	87	108	21	106	146	30	90	121	36	90	127	42	90	133	13	879	961
FETOX	17	479	500	21	490	530	30	482	513	36	482	519	42	482	525	13	1273	1355
FETOXM1	17	527	548	21	538	578	30	530	561	36	530	567	42	530	573	13	1321	1403
FGETEXP	17	27	48	21	38	78	30	30	61	36	30	67	42	30	73	13	821	903
FGETMAN	17	13	34	21	24	64	30	16	47	36	16	53	42	16	59	13	807	889
FINT	17	37	58	21	48	88	30	40	71	36	40	77	42	40	83	13	831	913
FINTRZ	17	37	58	21	48	88	30	40	71	36	40	77	42	40	83	13	831	913
FLOGN	17	507	528	21	518	558	30	510	541	36	510	547	42	510	553	13	1301	1383
FLOGNP1	17	553	574	21	564	604	30	556	587	36	556	593	42	556	599	13	1347	1429
FLOG10	17	563	584	21	574	614	30	566	597	36	566	603	42	566	609	13	1357	1439
FLOG2	17	563	584	21	574	614	30	566	597	36	566	603	42	566	609	13	1357	1439
FMOD	17	54	75	21	73	113	30	57	88	36	57	94	42	57	100	13	846	928
FMOVE to FPn	21	*	21	21	8	48	34	*	34	40	*	40	46	*	46	13	809	891
FMOVE to FPn**	10	0	21	21	8	48	28	6	34	34	6	40	40	6	46	13	809	891
FMOVE to memory*****	—	—	—	0	0	110	38	*	38	44	*	44	50	*	50	0	0	2006
FMOVE to memory**	—	—	—	0	0	110	0	0	38	0	0	44	0	0	50	0	0	2006
FMOVECR*****	10	0	32	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
FMUL	17	55	76	21	74	114	30	58	89	36	58	95	42	58	101	13	847	929
FNEG	17	17	38	21	28	68	30	20	51	36	20	57	42	20	63	13	811	893
FREM	17	84	105	21	103	143	30	87	118	36	87	124	42	87	130	13	876	958
FSCALE	17	25	46	21	44	84	30	28	59	36	28	65	42	28	71	13	817	899
FSGLDIV	17	53	74	21	72	112	30	56	87	36	56	93	42	56	99	13	845	927
FSGLMUL	17	43	64	21	62	102	30	46	77	36	46	83	42	46	89	13	835	917
FSIN	17	373	394	21	384	424	30	376	407	36	376	413	42	376	419	13	1167	1249
FSINCOS	17	433	454	21	444	484	30	436	467	36	436	473	42	436	479	13	1227	1309
FSINH	17	669	690	21	680	720	30	672	703	36	672	709	42	672	715	13	1463	1545
FSQRT	17	89	110	21	100	140	30	92	123	36	92	129	42	92	135	13	883	965
FSUB	17	35	56	21	54	94	30	38	69	36	38	75	42	38	81	13	827	909
FTAN	17	455	476	21	466	506	30	458	489	36	458	495	42	458	501	13	1249	1331
FTANH	17	643	664	21	654	694	30	646	677	36	646	683	42	646	689	13	1437	1519
FTENTOX	17	549	570	21	560	600	30	552	583	36	552	589	42	552	595	13	1343	1425
FTST	17	15	36	21	26	66	30	18	49	36	18	55	42	18	61	13	809	891
FTWOTOX	17	549	570	21	560	600	30	552	583	36	552	589	42	552	595	13	1343	1425

*These instruction do not have a tail time. The next instruction's head can be added to determine the effective head time.

**When register conflict occurs, concurrency is decreased.

***Add the effective address time to obtain overall execution time. Add the effective address time to obtain effective head time. (This does not apply to the FMOVE to memory instruction.)

****If the source or destination is an MPU data register, subtract five or two cycles, respectively.

*****Assumes a static k factor is used if the destination data format is packed decimal. Add 14 clock cycles if a dynamic k factor is used.

*****The source operand is from the constant ROM rather than a floating-point data register.

Table 8-4. Bus Cycle Activity — Arithmetic Operations

Operation Type	FPM Source	Memory Source or Destination Operand Format*				
		Integer**	Single**	Double	Extended	Packed
FPM Destination	(1/0/0/1/1)	(1/1/0/2/2)	(1/1/0/2/2)	(1/2/0/2/3)	(1/3/0/2/4)	(1/3/0/2/4)
Move to Memory***	—	(1/0/1/4/1)	(1/0/1/4/1)	(1/0/2/5/1)	(1/0/3/6/1)	(1/0/3/6/1)

*Add the appropriate effective address calculation bus cycle activity.

**If the source or destination is an MC68020 data register, subtract (0/1/0/0/0) or (0/0/1/0/0), respectively.

***Includes the read of one null (CA = 1, IA = 1) primitive when the conversion starts, the evaluate effective address and transfer data primitive when the conversion is complete, and a null (CA = 0) primitive after the transfer is complete. The MPU reads additional null (CA = 1, IA = 1) primitives while waiting for the transfer to start. For an MC68881, if no interrupts occur, the number of additional response CIR read cycles is 5 for integer, 3 for single or double, 1 for extended and ~194 for packed. For an MC68882, the number of additional response read cycles is 5 for integer and ~194 for packed.

Table 8-5. Timing Calculation Example

Instruction	<ea> Time	MC68881 Times	MC68882			
			Times	Adjusted	Effective Head Effective Tail	Actual Overlap
FMUL.D <ea>,FP1	6	98	95			
			H = 36	36 + 6 = 42		
			T = 58	58	58	58
FMOVE.D FP2,<ea>	6	86	44			
			H = 44	44 + 6 = 50	40 + 42 = 92	
			T = *	*		
FADD.D<ea>,FP1	6	78	75			
			H = 36	36 + 6 = 42		
			T = 38	38	38	38
FMOVE.X FP0,FP2	0	33	21			
			H = 21	21	21 + 42 = 63	
			T = *	*		
FMUL.D <ea>,FP2	6	98	95			
			H = 36	36 + 6 = 42		
			T = 58	58	59	58
FMOVE.D FP1,<ea>	6	86	44			
			H = 44	44 + 6 = 50	51 + 42 = 93	
			T = *	*		
FADD.D <ea>,FP2	6	78	75			
			H = 36	36 + 6 = 42		
			T = 38	38	38	
FMOVE.X FP0,FP1	0	33	21			
			H = 21	21	21	21
			T = *	*		
Total	36	557	470			175

overall 881 time: 557 + 36 = 593
 overall 882 time: 40 + 36 - 175 = 331
 ratio: 593/331 = 1.80

the corresponding times using the MC68882. The first column lists the instructions set. The second column lists the time required to obtain the operand at the effective address. These numbers are taken from Table 8-1. This time applies to both coprocessors; it is added to the execution time for the instruction. The third column lists the total execution times for each instruction when executing in the MC68881.

The four columns to the right list values that apply to the MC68882. From left to right, the columns contain the following values:

1. The overall execution time for each instruction when executing in the MC68882, and the H and T numbers for each of the instructions. The T values for the fully-concurrent FMOVE instructions are shown as T = *.
2. The adjusted head time, which is the sum of the effective address calculation time and the head time. For the FMOVE to memory instructions (opclass 011), the effective address calculation is not added to the head time. The tail time is not altered.
3. The effective head and effective tail time for each instruction. Where T is shown as T = *, the effective head time is the sum of the FMOVE H time plus the H time of the subsequent instruction. The tail time is not altered.
4. The actual overlap time, which is the lesser of the effective tail and the effective head of the column to the left.

At the bottom of the table, the totals show the overall times. For the MC68881, this time is the sum of the total execution time plus the effective address time. For the MC68882, it is the sum of the total execution time plus the effective address time, less the actual overlap time. The conclusion is that for the instruction sequence shown here, the MC68881 required 1.80 times longer to execute compared to the MC68882.

8.5.1.4 MOVE CONTROL REGISTER AND FMOVEM OPERATIONS. Table 8-6 shows the execution times for the FMOVE FPcr and FMOVEM instructions. The timing for the appropriate effective addressing mode must be added to the numbers in this table to determine the overall instruction execution times.

Table 8-6. Move Control Register and MOVEM Execution Times

Operation*		Best Case	Cache Case	Worst Case
FMOVE	FPcr,Rn	29/6 (0/0/0/3/1)	31/6 (0/0/0/3/1)	34/9 (1/0/0/3/1)
	FPcr,<ea>	31/6 (0/0/1/3/1)	33/6 (0/0/1/3/1)	36/9 (1/0/1/3/1)
	Rn,FPcr	26/6 (0/0/0/2/2)	28/6 (0/0/0/2/2)	31/9 (1/0/0/2/2)
	<ea>,FPcr	31/6 (0/1/0/2/2)	33/6 (0/1/0/2/2)	36/9 (1/1/0/2/2)
	#<data>,FPcr	30/6 (0/0/0/2/2)	30/6 (0/0/0/2/2)	31/9 (2/0/0/2/2)
FMOVEM	FPcr_list,<ea>	25 + 6n/6 (0/0/n/2 + n/1)	27 + 6n/6 (0/0/n/2 + n/1)	30 + 6n/9 (1/0/n/2 + n/1)
	<ea>,FPcr_list	25 + 6n/6 (0/n/0/2/1 + n)	27 + 6n/6 (0/n/0/2/1 + n)	30 + 6n/9 (1/n/0/2/1 + n)
	#<data>,FPcr_list	24 + 6n/6 (0/0/0/2/1 + n)	25 + 6n/6 (0/0/0/2/1 + n)	29 + 6n/9 (1 + n/0/0/2/1 + n)
	FPcr_list			
FMOVEM	FPdr_list,<ea>	35 + 25n/6 (0/0/3n/3 + 3n/1)	37 + 25n/6 (0/0/3n/3 + 3n/1)	40 + 25n/9 (1/0/3n/3 + 3n/1)
	<ea>,FPdr_list	33 + 31n/6 (0/3n/0/3/1 + 3n)	35 + 31n/6 (0/3n/0/3/1 + 3n)	38 + 31n/9 (1/3n/0/3/1 + 3n)
	Dn,<ea>	49 + 25n/6 (0/0/3n/4 + 3n/2)	51 + 25n/6 (0/0/3n/4 + 3n/2)	54 + 25n/9 (1/0/3n/4 + 3n/2)
	<ea>,Dn	47 + 31n/6 (0/3n/0/4/2 + 3n)	49 + 31n/6 (0/3n/0/4/2 + 3n)	52 + 31n/9 (1/3n/0/4/2 + 3n)

*Add the appropriate effective address calculation time. n is the number of registers transferred. Add two clocks if the coprocessor is an MC68882.

NOTE: FPcr or FPdr indicates any one of the floating-point control or data registers, respectively. FPcr_list or FPdr_list indicates a list of any combination of the floating-point control or data registers, respectively.

5 CONDITIONAL INSTRUCTIONS. Table 8-7 lists the execution times for the FPCP conditional instructions. Each entry in this table, except those for the FScc instruction, is complete and does not require the addition of values from any other table. For the FScc instruction, the only additional factor that must be included is the calculate effective address time for the operand to be modified.

Table 8-7. Conditional Instruction Execution Times

Operation	Comments	Best Case	Cache Case	Worst Case
FBcc.W	Branch Taken	18/6 (0'0'0'1/1)	20/6 (0'0'0'1/1)	23/6 (2'0'0'1'1)
	Branch Not Taken	16/6 (0'0'0'1/1)	18/6 (0'0'0'1/1)	19/6 (1'0'0'1'1)
FBcc.L	Branch Taken	18/6 (0'0'0'1/1)	20/6 (0'0'0'1/1)	23/6 (2'0'0'1'1)
	Branch Not Taken	16/6 (0'0'0'1/1)	18/6 (0'0'0'1/1)	21/6 (2'0'0'1'1)
FDBcc	True, Not Taken	18/6 (0'0'0'1/1)	20/6 (0'0'0'1/1)	24/9 (2'0'0'1'1)
	False, Not Taken	22/6 (0'0'0'1/1)	24/6 (0'0'0'1/1)	32/9 (4'0'0'1'1)
	False, Taken	18/6 (0'0'0'1/1)	20/6 (0'0'0'1/1)	26/9 (3'0'0'1'1)
FNOP	No Operation	16/6 (0'0'0'1/1)	18/6 (0'0'0'1/1)	19/6 (1'0'0'1'1)
FScc	Dn	16/6 (0'0'0'1/1)	18/6 (0'0'0'1/1)	21/9 (2'0'0'1'1)
	(An) + or - (An)*	18/6 (0'0'1'1/1)	22/6 (0'0'1'1/1)	25/9 (2'0'1'1'1)
	Memory**	16/6 (0'0'1'1/1)	20/6 (0'0'1'1/1)	23/9 (2'0'1'1'1)
FTRAPcc	Trap Taken	36/6 (0'1/4'1/1)	39/6 (0'1/4'1/1)	47/9 (3'1'4'1'1)
	Trap Not Taken	16/6 (0'0'0'1/1)	18/6 (0'0'0'1/1)	22/9 (2'0'0'1'1)
FTRAPcc.W	Trap Taken	38/6 (0'1/4'1/1)	41/6 (0'1/4'1/1)	45/9 (3'1'4'1'1)
	Trap Not Taken	18/6 (0'0'0'1/1)	20/6 (0'0'0'1/1)	23/9 (2'0'0'1'1)
FTRAPcc.L	Trap Taken	40/6 (0'1/4'1/1)	43/6 (0'1'4'1'1)	52/9 (4'1'4'1'1)
	Trap Not Taken	20/6 (0'0'0'1/1)	22/6 (0'0'0'1/1)	27/9 (3'0'0'1'1)

*For condition true; subtract one clock for condition false.

**Add the appropriate effective address calculation time.

Since the conditional instructions are intrinsic to the M68000 Family coprocessor interface (i.e., they are not defined by the FPCP through the use of response primitives), the MPU performs most of the processing associated with these instructions. The only part of the instruction that the FPCP performs is the evaluation of the condition predicate written to the condition CIR. Thus, the execution times shown in Table 8-7 are heavily dependent on the environment in which the main processor executes.

The overlap allowed times listed for these instructions indicate the time at the beginning of the instruction that can overlap with the execution of the previous instruction by the FPCP. No overlap is allowed at the end of the instruction since the FPCP is always idle while the MPU is completing the operation.

8.5.1.6 FSAVE AND FRESTORE INSTRUCTIONS. The time required for a context save or restore operation is shown in Table 8-8. The appropriate calculate effective address times must be added to the values in this table to obtain the total execution time for these operations. For the FSAVE instruction, the FPCP may use the not-ready format code to force the MPU to wait while internal operations are completed in order to reduce the size of the saved state frame or reach a point where a save operation can be performed. The idle time occurs if the FPCP is in the idle phase when the save CIR is written (refer to 6.4.3 **FSAVE Protocol** and 6.4.4 **FRESTORE Protocol** for definitions of instruction phases). The busy time occurs if the FPCP is in the initial phase or at a save boundary in the middle

Table 8-8. FSAVE and FRESTORE Instruction Execution Times

Coprocessor	Operation	State Frame	Best Case	Cache Case	Worst Case
MC68881	FRESTORE	Null	19/4* (0/1/0/1/1)	21/4* (0/1/0/1/1)	22/4* (1/1/0/1/1)
		Idle	55/4* (0/7/0/1/7)	57/4* (0/7/0/1/7)	58/4* (1/7/0/1/7)
		Busy	289/4* (0/46/0/1/46)	291/4* (0/46/0/1/46)	292/4* (1/46/0/1/46)
MC68881	FSAVE	Null	14/1 (0/0/1/1/0)	16/1 (0/0/1/1/0)	18/1 (1/0/1/1/0)
		Idle	50/1 (0/0/7/7/0)	52/1 (0/0/7/7/0)	54/1 (1/0/7/7/0)
		Busy	284/1 (0/0/46/46/0)	286/1 (0/0/46/46/0)	288/1 (1/0/46/46/0)
MC68882	FRESTORE	Null	19/4* (0/1/0/1/1)	21/4* (0/1/0/1/1)	22/4* (1/1/0/1/1)
		Idle	103/4* (0/15/0/1/15)	105/4* (0/15/0/1/15)	106/4* (1/15/0/1/15)
		Busy	337/4* (0/54/0/1/54)	339/4* (0/54/0/1/54)	340/4* (1/54/0/1/54)
MC68882	FSAVE	Null	14/1 (0/0/1/1/0)	16/1 (0/0/1/1/0)	18/1 (1/0/1/1/0)
		Idle	98/1 (0/0/15/15/0)	100/1 (0/0/15/15/0)	102/1 (1/0/15/15/0)
		Busy	332/1 (0/0/54/54/0)	334/1 (0/0/54/54/0)	336/1 (1/0/54/54/0)

*Add the appropriate effective address calculation time. Note that the overlap time available for the FRESTORE instruction is of little use, since this operation destroys the previous context of the FPCP.

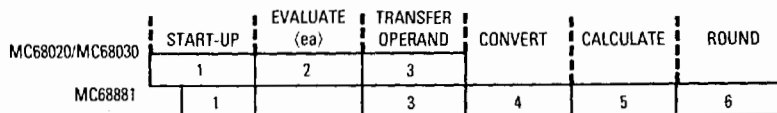
**The second overlap allowed number represents the period during which the FPCP is preparing to perform the save operation and the MPU can process interrupts.

8.5.2 MC68881 Detail Timing Tables

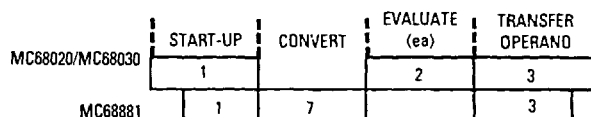
This set of tables provides the information needed to calculate a more precise execution time for an instruction executing in the MC68881, based on the input operand format and type, than can be obtained with the typical timing tables shown previously. Also, these tables contain the information necessary to determine instruction execution timing for a system that does not utilize the MPU as the main processor. The assumptions stated previously are used for these tables, with further restrictions described separately for each table. Note that the timing numbers in the typical timing tables are derived, in most cases, by using the following set of tables.

In order to better understand the relationship of each table in this group, the following diagrams are included. These diagrams break each basic instruction type into separate execution components. For each component, the appropriate tables that are used to calculate the execution time are identified. These diagrams can also be used to clarify the distribution of responsibility for instruction execution between the MPU and the MC68881 and to more clearly illustrate the periods of time during which overlapped execution may occur. In these diagrams, the numbers inside each box indicate the table that is used to determine the timing for that phase of the instruction; the identification key for these tables

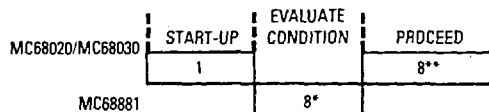
Memory-to-Register, Register-to-Register Operations



Move Register-to-Memory



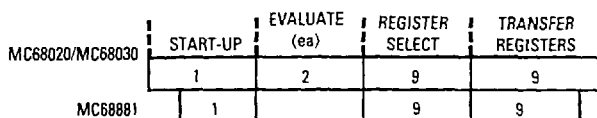
Conditional Operations



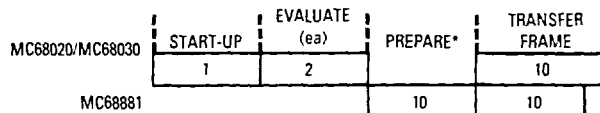
*The timing for the evaluation of the conditional predicate is shown separately in **8.5.2.7 CONDITIONAL TERMINATION**.

**The action taken by the MC68020/MC68030 after the conditional predicate is evaluated by the – xCCw on the instruction (FPcc, FDBcc, FScc, or FTRAPcc).

Move Control or Multiple Registers

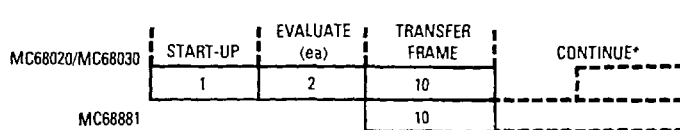


Context Save Operation



*During this period, the MC68881 may force the MC68020/MC68030 to wait while an internal operation is completed, or reaches a point where a save operation can be performed.

Context Restore Operation



*When the context restore operation is completed, the MC68881 continues with any operation that was suspended by a previous context save. The MC68020/MC68030 does not re-establish communications with the MC68881 during the FRESTORE instruction, but the execution of a subsequent RTE instruction restores the MC68020/MC68030 context to the state of the previously suspended operation if necessary.

e Identification:

- Instruction Start-Up
- 2 — Effective Address Calculations
- 3 — Operand Transfers
- 4 — Input Operand Conversions
- 5 — Arithmetic Calculations
- 6 — Rounding and Exception Handling
- 7 — Output Operand Conversions
- 8 — Conditional Instructions
- 9 — Multiple Register Transfers
- 10 — State Frame Transfers
- 11 — Exception Processing

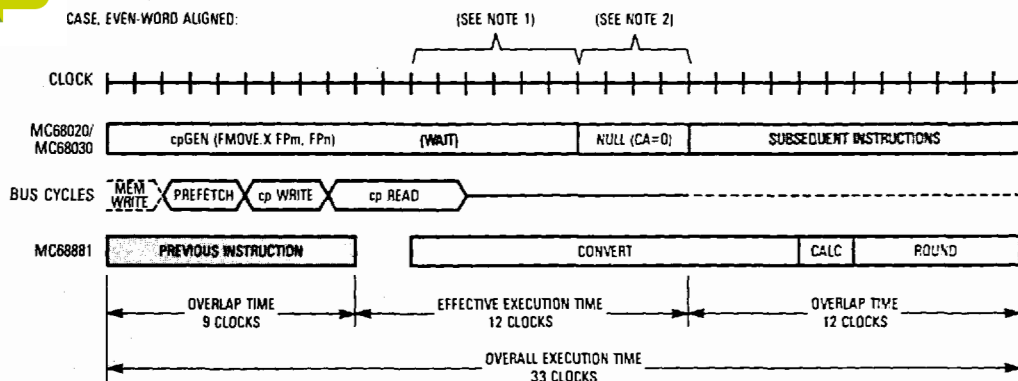
As an example of the use of the information in the following paragraphs, consider the FADD.P (A0)+,FP0 instruction. First, the instruction start-up table is used to determine the time required by the MPU to initiate the instruction (by writing the command word and reading the first response primitive). In this case, the first response is evaluate effective address and transfer data (with the PC bit set if any exceptions are enabled). The operand transfer table is then used to determine the time required to transfer the packed decimal string from memory to the MC68881, and this table requires the addition of the effective address calculation time. Thus, the calculate effective address table is used to determine the time required by the MPU to calculate the effective address, (A0)+, and those numbers are added to the start-up and transfer timing numbers. Note that these first three values are almost entirely dependent on the MPU and do not apply if the main processor is not an MC68020 or MC68030.

To complete the timing calculation, a fourth table is used to determine the decimal-to-binary conversion time, based on the input operand data type and value. Finally, the fifth and sixth tables used determine the time required for the addition and rounding operations. The second set of three operations are totally independent of the main processor, and timing numbers derived for them can be utilized by non-MPU based system designers.

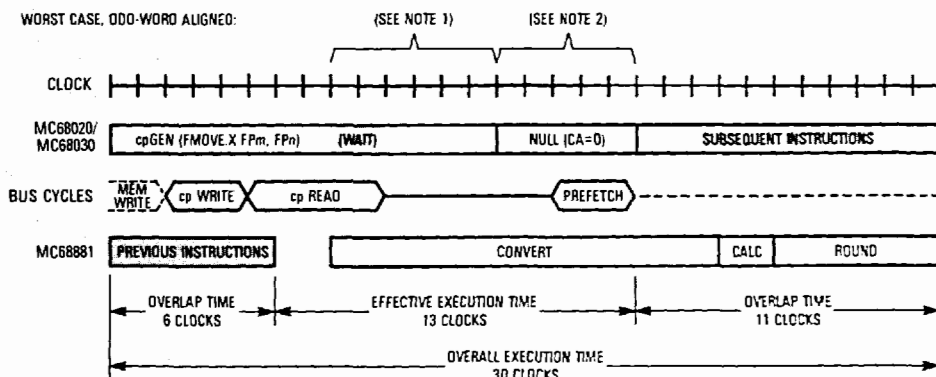
As a further aid to understanding the interaction of the MPU with the MC68881 during the execution of an instruction, four diagrams are presented in Figures 8-4 and 8-5. The bus cycle activity and overlapped execution that is allowed during the communications dialog is shown in the diagrams, in addition to illustrations of the effect of instruction alignment, enabled exceptions, and device synchronization. These diagrams represent the clock-cycle-by-clock-cycle activity of the two devices for four cases of the FMOVE instruction. The first three diagrams describe the FMOVE.X Fp_m,Fp_n instruction for worst-case and cache-case operation, and the fourth diagram describes the FMOVE.X (An),Fp_n instruction.

The three diagrams in Figure 8-4 show three cases of the FMOVE.X Fp_m,Fp_n instruction. The first and second cases show worst-case operation (where the instruction prefetches required to replace the FMOVE instruction do not hit in the MPU on-chip cache) for the two possible alignments of the instruction. If the first word of the instruction is at an even word address, the prefetch request generated by the cpGEN start-up operation (to replace the F-line operation word) causes an external bus cycle to be executed. This prefetch acquires two words, one of which fills the cpGEN request, and one that is held in a temporary register. The time required to execute this prefetch cycle adds directly to the overall execution time for the instruction, as well as the front-end overlap allowed time. When the null (CA=0) primitive is processed by the MPU, a second prefetch request is generated (to replace the command word which is filled with the word from the temporary register). Thus, the null operation prefetch request does not generate an external bus cycle.

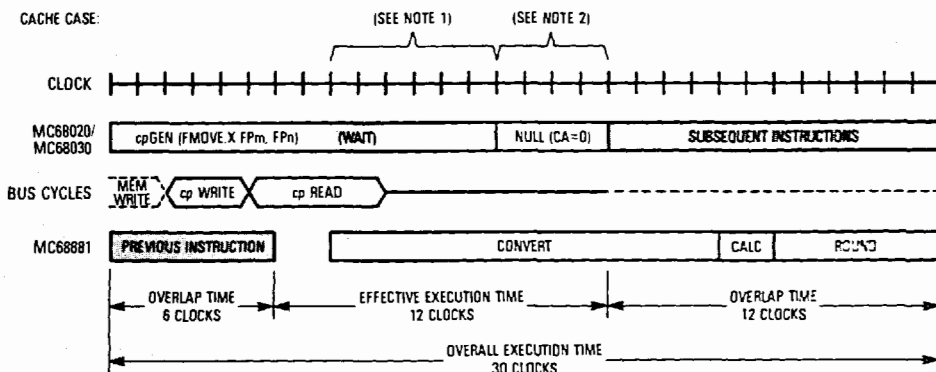
CASE, EVEN-WORD ALIGNED:



WORST CASE, ODD-WORD ALIGNED:



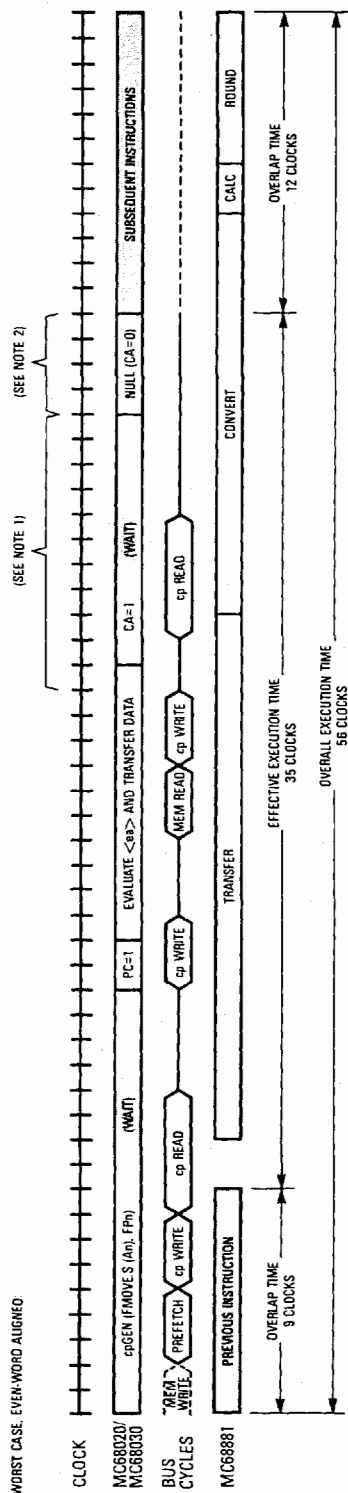
CACHE CASE:



NOTES:

1. These six clocks do not add to the overall execution time for the instruction, only the effective execution time for the MC68020.
2. This operation does not add to the overall execution time for the instruction, only the effective execution time for the MC68020.

Figure 8-4. Instruction Overlap Examples — FMOVE.X FpM,FpN



NOTES:

1. These eleven clocks do not add to the overall execution time for the instruction, only the effective execution time for the MC68020.
2. This operation does not add to the overall execution time for the instruction, only the effective execution time for the MC68020.

Figure 8-5. Instruction Overlap Example — FMOVE.S (An),FPn

When the MPU polls the response CIR, the MC68881 begins execution of the instruction during the fourth clock cycle of the read cycle. As the MC68881 proceeds with the conversion operation, the MPU then completes the cpGEN start-up operation and processes the null (CA=0) primitive. The 10 clock cycles required to perform these operations overlap with the execution of the instruction by the MC68881 and, thus, are not included in the overall execution time calculation (although they are included in the effective execution time calculation). The same consideration applies to the second and third diagrams in Figure 8-4.

As shown in the second diagram of Figure 8-4, if the first word of the instruction is at an odd word address, the prefetch requested by the cpGEN start-up is filled from the temporary register (which was loaded by a prefetch requested during the previous MPU instruction) and an external bus cycle is not required. When the null (CA=0) primitive is processed, a second prefetch request is generated which must be filled by the execution of an external bus cycle. Thus, the start-up operation for this case is a minimum of three clock cycles shorter than the first case (although the overlap allowed time is also shorter) while the time required to process the null primitive is at least one clock cycle longer. Since the null processing overlaps with the execution of the operand conversion by the MC68881, the overall execution time for the instruction is shorter, although the overlap allowed time at the end of the instruction is reduced.

For the third case, both of the instruction prefetch requests generated during the instruction execution are satisfied by either the temporary register or the on-chip instruction cache. Thus, the overall execution time achieves the absolute best case while allowing the maximum possible overlap between the two devices.

The diagram in Figure 8-5 illustrates the execution of the FMOVE.S (An),FPn instruction where the instruction is even-word aligned, the MPU cache is disabled, and at least one of the arithmetic exceptions is enabled. Under these conditions, the cpGEN start-up operation is identical to the first diagram in Figure 8-4, except that the primitive returned by the MC68881 is evaluate effective address and transfer data with the PC and CA bits set. Thus, the first operation performed by the MPU while processing this primitive is to pass the program counter, which adds two clock cycles to both the effective and overall execution times. (Note that the third clock cycle of the coprocessor write cycle overlaps with the effective address calculation.) The MPU then evaluates the effective address, (An), which requires two clock cycles, and transfers the 32-bit single precision operand from memory. The come-again operation is then performed, which requires 10 clock cycles, followed by a four clock period during which the null (CA=0) primitive is processed.

The MC68881 does not start the input conversion operation until the single precision operand is internally passed to the execution unit. The MC68881 bus interface unit requires three clocks from the end of the operand write cycle to transfer the operand to the execution unit; thus, the conversion does not begin until three clock cycles after the end of the write cycle. This three-clock-cycle transfer operation and part of the conversion operation occur simultaneously with the completion of the CA=1 and null processing by the MPU. Thus, 15 clock cycles of the MPU effective execution time do not contribute to the overall execution time for the instruction.

The previous four examples are intended to clarify the meaning of the detailed execution timing tables that follow. The only difference between the FMOVE instruction examples presented and any of the monadic or dyadic instructions is that the convert and calculate times are different. (The round time is also different if an exception occurs.) Also, the

Effective address calculation and operand transfer times are different. Notice that the timing to the start of the conversion operation is almost entirely dependent on the execution characteristics of the main processor, while the timing for the rest of the instruction is dependent solely on the FPCP. This distinction is useful when the execution timing for a main processor other than the MC68020 or MC68030 is to be determined.

NOTE

The term "not normalized" is used frequently in the following tables. This term is used where conditions allow the input of a denormalized or unnormalized number, and the term "denormalized" is used where only a denormalized input is possible. Refer to **3.2.2 Denormalized Numbers** for a description of the denormalized and unnormalized data types.

8.5.2.1 INSTRUCTION START-UP. When the MPU encounters an FPCP instruction, it decodes the type of the coprocessor instruction and then initiates communications with the FPCP using the appropriate coprocessor interface bus cycle. Table 8-9 lists the execution timing of the MPU for the start-up phase of each of the coprocessor instruction types. For the general instruction type, the start-up time includes the command CIR write and response CIR read cycles that initiate the instruction dialog between the MPU and the FPCP. For the conditional instruction types, the start-up time includes the condition CIR write and response CIR read cycles.

Table 8-9. Instruction Start-Up Times

Instruction Type	Best Case	Cache Case	Worst Case
General*	12/6 (0/0/0/1/1)	14/6 (0/0/0/1/1)	17/9 (1/0/0/1/1)
FBcc	12/6 (0/0/0/1/1)	14/6 (0/0/0/1/1)	14/6 (0/0/0/1/1)
FDBcc, FSec and FTRAPcc	12/6 (0/0/0/1/1)	14/6 (0/0/0/1/1)	17/9 (1/0/0/1/1)
FSAVE**	13/1 (0/0/1/1/0)	15/1 (0/0/1/1/0)	15/1 (0/0/1/1/0)
FRESTORE**	16/4** (0/1/0/1/1)	18/4** (0/1/0/1/1)	18/4** (0/1/0/1/1)

*These execution time numbers represent the overall execution time for this operation with respect to the MPU, and therefore, are used to calculate the effective execution time of the instruction. However, six clock cycles always overlap with the execution of a register-to-register instruction (OPCLASS 000) by the FPCP, and therefore, should not be included in the calculation to generate the overall execution time.

**Add the appropriate effective address calculation time. Note that the overlap time available for the FRESTORE instruction is of little use, since this operation destroys the previous context of the FPCP.

For the FSAVE instruction, the start-up time includes the read of the save CIR and the write of the format word to memory. For the FRESTORE instruction, the start-up time includes the read of the format word from memory, the write of the restore CIR, and the read of the restore CIR to validate the format word. The effective address calculation time is not included for the FSAVE and FRESTORE instructions; the appropriate values must be obtained from the calculate effective address table and added to the start-up values for these instructions.

If an enabled pre-instruction exception is pending when the MPU attempts to initiate an FPCP instruction, the instruction start-up operation is performed for the general or conditional instruction types, and then the MPU proceeds to perform exception processing (at

request of the FPCP). In this case, the start-up timing numbers are added to the values in the exception processing tables to determine the time required to begin execution of the exception handler.

The MPU terminates all instructions except FSAVE and FRESTORE by processing a null (CA=0) primitive (unless a mid-instruction exception occurs). Therefore, the timing values in Table 8-10 should be included in the calculation of the effective execution time for the MPU, where appropriate.

Table 8-10. Null Primitive Time Values

Primitive Type	Best Case	Cache Case	Worst Case
Null (CA=0) with no tracing	4* (0 0 0 0 0)	4 4* (0 0 0 0 0)	5 5* (1 0 0 0 0)

*Overlap is allowed for register-to-register and external-to-register instructions only (OPCLASS 000 and 010).

8.5.2.2 TRANSFER OPERAND. Tables 8-11 and 8-12 show the timing for the transfer of an operand to or from the FPCP by the MPU. Table 8-11 shows the values for external source or destination operands that reside in an MPU register or in memory, and Table 8-12 shows the values for immediate source operands. For input transfers, the timing numbers shown include the time required by the MPU to process the evaluate effective address and transfer data (with CA=1) primitive, and for the FPCP to perform the internal transfer of the operand to the execution unit. For the MPU, the last clock cycle of the transfer operation and the processing for CA=1 always overlaps with the input operand transfer and conversion operations by the FPCP, and therefore, is not added to the overall execution time for the instruction (although these operations are included in the calculation of the effective execution time for the MPU).

Table 8-11. Operand Transfer Time — External Operand

Transfer Type	Operand Format				
	Byte	Word	Long, Single	Double	Ext., Packed
From MC68020 Dn	14/0 (0/0/0/1/1)	14/0 (0/0/0/1/1)	14 0 (0 0 0 1 1)	—	—
From Memory*	19/0 (0/1/0/1/1)	19/0 (0/1/0/1/1)	19 0 (0 1 0 1 1)	25 0 (0 2 0 1 2)	31 0 (0 0 3 1 0)
To MC68020 Dn	17/0 (0/0/0/3/0)	17/0 (0/0/0/3/0)	17 0 (0 0 0 3 0)	—	—
To Memory**	19/0 (0/0/1/3/0)	19/0 (0/0/1/3/0)	19 0 (0 0 1 3 0)	25 0 (0 0 2 4 0)	31 0 (0 0 3 5 0)

*Add the appropriate effective address calculation time. Eleven clocks of the MPU processing overlap with execution by the MC68881, which requires five or three clock cycles after the last coprocessor write cycle to complete the internal transfer for double or any other format, respectively. Thus, reduce the numbers above by six clocks for double or eight clocks for any other format for calculation of the overall execution time.

**Add the appropriate effective address calculation time. In the event the destination is packed decimal and a dynamic k factor is used, add 14/0 (0/0/0/1/1).

Table 8-12. Operand Transfer Time — Immediate Operand

Immediate Operand Format	Best Case	Cache Case	Worst Case
Byte, Word	14/0 (0 0 0 1 1)	14 0 (0 0 0 1 1)	17 0 (1 0 0 1 1)
Long, Single	18/0 (0 0 0 1 1)	18 0 (0 0 0 1 1)	19 0 (1 0 0 1 1)
Double	22/0 (0 0 0 1 2)	22 0 (0 0 0 1 2)	24 0 (2 0 0 1 2)
Extended, Packed	26/0 (0 0 0 1 3)	26 0 (0 0 0 1 3)	30 0 (3 0 0 1 3)



output operand transfers, the timing numbers include the processing for the evaluate effective address and transfer data primitive (with CA = 1). Since no overlap occurs during an output transfer, the values in the table are used directly in the overall execution time calculation. Note that the bus cycle activity numbers include the read of the evaluate effective address and transfer data primitive at the end of the conversion (even though the execution time for the conversion is not included). The read time is included because null (CA = 1, IA = 1) primitives are read during the instruction start-up operation and while waiting for the conversion to complete, and the evaluate effective address and transfer data primitive is read during the processing of one of those primitives.

In order to calculate the effective execution time for the MPU for either input or output transfers, the processing time for the null (CA = 0) primitive that terminates the dialog must be included. For output conversions that cause an enabled exception, the take mid-instruction exception primitive is returned after the operand transfer is complete. In this case, the appropriate exception processing execution time values must be included in lieu of the null (CA = 0) processing time in the calculation of the overall execution time.

8.5.2.3 INPUT OPERAND CONVERSION. All FPCP instructions that require an input operand execute an implied conversion to the 80-bit extended precision format that is used internally. The amount of time required to perform this conversion depends on the format, value, and type of the input operand. Table 8-11 shows the amount of time required to convert an input operand to the internal data format, starting from the end of the internal operand transfer after the last write cycle to the operand CIR.

For dyadic operations, one portion of Table 8-13 for conversions from each combination of source data format and type versus destination data type is included. For monadic operations, one portion includes the conversion timing for any data format and type. Only one number is listed in each entry, since the total number of clock cycles required is equal to the number of overlap allowed clock cycles, and no bus cycles are generated during this stage of an instruction (since the FPCP does not require any further services of the MPU after this stage of an instruction starts).

8.5.2.4 ARITHMETIC CALCULATION. Tables 8-14 and 8-15 show the time required by the MC68881 to perform any of its general-purpose arithmetic operations. One portion of Table 8-14 shows the execution time values for each dyadic instruction with respect to the combination of input operand data types. Table 8-15 shows the execution time values for all of the monadic operations. Each entry in these tables includes the time from the end of the input operand conversion to completion of the calculation. Only one number is shown for each entry, since no bus cycles are generated during this stage of an instruction. Also, the total number of clock cycles required for the calculation is equal to the number of overlap allowed clock cycles, since the FPCP does not require any further services of the MPU after this stage of an instruction starts.

Some entries in these tables refer to a footnote that contains more detailed timing information for an operation (e.g., the table for addition contains an entry that references the ADD footnote, which contains three numbers, based on the input operands). Furthermore, in some cases, an entry refers to another table that contains the execution time required to handle certain input operands. For example, if an entry contains NAN1, refer to the entry of the same name in **8.5.2.10 EXCEPTION PROCESSING**.

Dyadic Input Conversions — Source Operand is Byte, Word, or Long:

Destination \ Source	Normalized		Not Normalized	Zero	Infinity	NAN
	+	-				
Normalized	24	26	—	22	—	—
Unnormalized	36	38	—	34	—	—
Zero	30	32	—	28	—	—
Infinity	28	30	—	26	—	—
NAN	30	32	—	28	—	—

Dyadic Input Conversions — Source Operand is Single Precision:

Destination \ Source	Normalized		Not Normalized	Zero	Infinity	NAN
	+	-				
Normalized	18		36	22	24	26
Unnormalized	30		48	34	36	38
Zero	24		42	28	30	32
Infinity	22		40	26	28	30
NAN	24		42	28	30	32

Dyadic Input Conversions — Source Operand is Double Precision:

Destination \ Source	Normalized		Not Normalized	Zero	Infinity	NAN
	+	-				
Normalized	16		34	20	22	24
Unnormalized	28		46	32	34	36
Zero	22		40	26	28	30
Infinity	20		38	24	26	28
NAN	22		40	26	28	30

Dyadic Input Conversions — Source Operand is Extended Precision:

Destination \ Source	Normalized		Not Normalized	Zero	Infinity	NAN
	+	-				
Normalized	10		26	12	12	14
Unnormalized	22		38	24	24	26
Zero	16		32	18	18	20
Infinity	14		30	16	16	18
NAN	16		32	18	18	20

Monadic or Dyadic Input Conversions — Source Operand is Packed Decimal:

Destination \ Source	Normalized	Not Normalized	Zero	Infinity	NAN
Normalized	~822	~822	22	22	24
Unnormalized	~848	~848	34	34	36
Zero	~842	~842	28	28	30
Infinity	~840	~840	26	26	28
NAN	~842	~842	28	28	30

~Indicates a typical conversion time. The minimum maximum conversion time is 954 clock cycles.

Monadic or Dyadic Input Conversions — Source Operand is FPM:

Destination \ Source	Normalized	Not Normalized	Zero	Infinity	NAN
Normalized	14	30	16	16	18
Unnormalized	26	42	28	28	30
Zero	20	36	22	22	24
Infinity	18	34	20	20	22
NAN	20	36	22	22	24

Monadic Input Conversions — Source Operand is in Memory:

Format \ Type	Normalized + -	Not Normalized	Zero	Infinity	NAN
Byte, Word, Long	22 24	—	20	—	—
Single	16	30	20	24	24
Double	14	28	18	22	22
Extended	8	20	10	12	12

Table 8-14. Arithmetic Calculation Times — Dyadic Operations

FADD Calculation Time:

Destination \ Source		Normalized		Zero		Infinity		NAN	
		+	-	+	-	+	-	+	-
Normalized	+	ADD		2+		6		NAN2	
	-								
Zero	+	2+		6 26		6		NAN2	
	-								
Infinity	+	6		6		6 20		20 6	
	-								
NAN	+	NAN1		NAN2		NAN2		NAN3	
	-								

ADD: 24+ if the source and destination exponents are equal, and the source mantissa is less than the destination mantissa.
 26+ if the source and destination exponents are equal, and the source mantissa is greater than or equal to the destination mantissa.
 28+ if the source and destination exponents are not equal.

FCMP Calculation Time:

8

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	CMP 6	6	6	8	6	NAN4	NAN4
	-	6 CMP			6	8		
Zero	+	8 6	6	6	8	6	NAN4	NAN4
	-	6 8			6	8		
Infinity	+	6	6	6	6	6	NAN4	NAN4
	-							
NAN	+	NAN1	NAN2	NAN2	NAN2	NAN2	NAN3	NAN3
	-							

CMP: 8 if the source exponent is greater than the destination exponent.
 10 if the source exponent is less than or equal to the destination exponent.

FDIV Calculation Time:

Destination \ Source		Normalized		Zero		Infinity		NAN	
		+	-	+	-	+	-	+	-
Normalized	+	DIV		20		6 8		8 6	
	-								
Zero	+	6 8		20		6 8		NAN2	
	-								
Infinity	+	6 8		6 8		20		NAN2	
	-								
NAN	+	NAN1		NAN2		NAN2		NAN3	
	-								

DIV: 78+ if the intermediate result is normalized.
 80+ if the intermediate result is denormalized.

FMOD Calculation Time:

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	-	MOD		20		6+	NAN2
Zero	+	-	6+		20		6+	NAN2
Infinity	+	-	IOP		20		20	NAN2
NAN	+	-	NAN1		NAN2		NAN2	NAN3

MOD: 18+ if the quotient is zero; else:
 $(40 + 70 \cdot \text{INT}(1 + \text{destination exponent} - \text{source exponent})) +$
 $(\quad \quad \quad)$

FMUL Calculation Time:

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	-	MUL		6 8		6 8	NAN2
Zero	+	-	6	8	6	8	20	NAN2
Infinity	+	-	6	8	20		6	8
NAN	+	-	NAN1		NAN2		NAN2	NAN3

MUL: 46+ if the intermediate result is normalized.
 48+ if the intermediate result is not normalized.

FREM Calculation Time:

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	-	REM		20		6+	NAN2
Zero	+	-	6+		20		6+	NAN2
Infinity	+	-	IOP		20		20	NAN2
NAN	+	-	NAN1		NAN2		NAN2	NAN3

REM: 18+ if the quotient is zero; else:
 $(40 + 70 \cdot \text{INT}(1 + \text{destination exponent} - \text{source exponent})) +$
 $(\quad \quad \quad 64 \quad \quad \quad)$

FSCALE Calculation Time:

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	-	SCALE		6 +		20	
Zero	+	-	6		6		20	
Infinity	+	-	6		6		20	
NAN	+	-	NAN1		NAN2		NAN2	
							NAN3	

SCALE: 12+ if the source exponent (unbiased) is less than zero.
 16+ if the source exponent (unbiased) is in the range [0 ... 15].
 20+ if the source exponent (unbiased) is greater than 15.

FSGLDIV Calculation Time:

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	-	SGLDIV		20		6 8 8 6	
Zero	+	-	6 8		20		6 8	
Infinity	+	-	6 8		6 8		20	
NAN	+	-	NAN1		NAN2		NAN2	
							NAN3	

SGLDIV: 44 if no extended precision underflow or overflow occurs.
 62 if an extended precision overflow occurs.
 90 if an extended precision underflow occurs.

FSGLMUL Calculation Time:

Destination \ Source	Normalized		Zero		Infinity		NAN	
	+	-	+	-	+	-	+	-
Normalized	+	-	SGLMUL		6 8 8 6		20	
Zero	+	-	6 8		6 8		20	
Infinity	+	-	6 8		20		6 8	
NAN	+	-	NAN1		NAN2		NAN2	
							NAN3	

SGLMUL: 34 if no extended precision underflow or overflow occurs.
 52 if an extended precision overflow occurs.
 80 if an extended precision underflow occurs.

FSUB Calculation Time:

Destination \ Source		Normalized		Zero		Infinity		NaN	
		+	-	+	-	+	-	+	-
Normalized	+	SUB		2+		8		NaN2	
	-								
Zero	+	4+		26 8		8		NaN2	
	-								
Infinity	+	6		6		20 8		8 20	
	-								
NaN	+	NaN1		NaN2		NaN2		NaN3	
	-								

SUB: 24+ if the source and destination exponents are equal, and the source mantissa is less than the destination mantissa.
 26+ if the source and destination exponents are equal, and the source mantissa is greater than or equal to the destination mantissa.
 28+ if the source and destination exponents are not equal.

If an entry in these tables is appended with a plus sign (+), the appropriate timing numbers from the rounding and exception handling table (in **8.5.2.10 EXCEPTION PROCESSING**) must be used to calculate the overall execution time for an instruction.

Otherwise, the numbers from these tables include the time to handle exceptional operand cases and produce the final result.

8.5.2.5 OUTPUT OPERAND CONVERSION. The FMOVE.<fmt> FPn,<ea> instruction performs an implicit conversion from the 80-bit extended precision format used internally by the FPCP to an external data format. Table 8-16 lists the conversion times for most output operations. Since the execution timing for conversions from the internal extended precision format to either single or double precision is highly data dependent, the timing for these operations (for in-range, nonzero input values) is listed in a second table, Table 8-17.

The amount of time required to perform this conversion depends on the value and type of the input operand and the format of the desired output. The values given in the following tables, in FPCP clock cycles, include the time from the fourth clock cycle of the first response CIR read (which returns a null (CA = 1, IA = 1) primitive) to completion of the conversion (when a read of the response CIR returns an evaluate effective address and transfer operand primitive). Only one number is shown for each entry, since no bus cycles are generated during this stage of an instruction. Also, the total number of clock cycles required for operand conversion is equal to the number of overlap allowed clock cycles (during which time interrupts may be handled; normal program execution is not allowed), since the FPCP does not require any services of the MPU during this stage of an instruction.

8.5.2.6 ROUNDING AND EXCEPTION HANDLING. Tables 8-18 and 8-19 contain the execution times for rounding and for various exception handling operations. For the typical execution time tables shown previously, it is assumed that the MC68881 uses the default operating mode of round-to-extended precision, and no overflow or underflow exceptions occur. If this is not the case, the round/store phase of most arithmetic instructions takes longer to execute. The entries in the typical execution time tables include the processing time for no underflow, overflow, or round overflow as indicated in Table 8-18.

Table 8-15. Arithmetic Calculation Times — Monadic Operations

Operation	Source	Normalized		Zero		Infinity		NaN	
		+	-	+	-	+	-	+	-
FABS		4+		4+		8		NAN2	
FACOS		594+		12 ¹ 20 ²		20		NAN2	
FASIN		550+		6		20		NAN2	
FATAN		372+		6		12 ¹ 20 ²	14 ¹ 22 ²	NAN2	
FATANH		662+		6		20		NAN2	
FCOS		360+3		8		20		NAN2	
FCOSH		576+		8		8		NAN2	
FETOX		466+		8		6		NAN2	
FETOXM1		514+		6		6	8	NAN2	
FGETEXP		exponent = 0:16 exponent > 0:20 exponent < 0:22		6		20		NAN2	
FGETMAN		6		6		20		NAN2	
FINT, FINTRZ		fraction = 0:8 fraction ≠ 0:30 result = 0:28		6		60		NAN2	
FLOGN		494+ IOP		22		6	20	NAN2	
FLOGNP1		540+ ⁴		6		6	20	NAN2	
FLOG10		550+ IOP		22		6	20	NAN2	
FLOG2		550+ IOP		22		6	20	NAN2	
FMOVE to FPn		2+		6		6		NAN2	
FMOVECR		18 ¹ 26 ²		—		—		—	
FNEG		4+		4+		8		NAN2	
FSIN		360+ ³		6		20		NAN2	
FSINCOS		420+ ³		20		26		NAN6	
FSINH		656+		6		6		NAN2	
FSQRT		76+ IOP		6		6	20	NAN2	
FTAN		442+ ³		6		20		NAN2	
FTANH		630+		6		8		NAN2	
FTENTOX		536+		8		6		NAN2	
FTST		8		8		8		NAN5	
FTWOTOX		536+		8		6		NAN2	

NOTES:

1. If the extended precision rounding mode is used.
2. If the single or double precision rounding mode is used.
3. This assumes that the source operand is in the range (-9...+9). If the source operand is outside of that range, the appropriate REM calculation time required to perform the argument reduction must be added to this value.
4. If the source operand is less than or equal to -1, use the IOP time.

Table 8-16. Output Operand Conversion

Dest. Format \ Source Type	Normalized + -		Not Normalized		Zero	Infinity	NAN
Integer, No Overflow	50	52	60	62	18	—	24
Integer, Overflow	52	56	62	66	18	24 26	24
Single	(see below)		(see below)		16	18	NAN7
Double	(see below)		(see below)		16	18	NAN7
Extended	18		(see note 1)		16	16	NAN7
Packed	(see note 2)		(see note 2)		24	24	NAN2

NOTES:

1. 26 clocks if the source operand is an unnormalized number. 56 clocks if the source operand is a denormalized number.
2. 1942 clocks is the typical time required for the conversion, if no overflow occurs. The maximum time is 3674 clocks.

Table 8-17. Output Operand Conversion — Binary Real Formats

Conversion Result \ Source Type	Normalized	Not Normalized
No Underflow, Overflow or Round Overflow	38	48
No Underflow or Overflow; Round Overflow	42	52
Overflow; RN or RZ Mode; No Round Overflow	44	54
Overflow; RN or RZ Mode; Round Overflow	48	58
Overflow; RM or RP Mode; No Round Overflow	46	56
Overflow; RM or RP Mode; Round Overflow	50	60
Underflow; No Round Overflow	66	76
Underflow; Round Overflow	70	80

Table 8-18. Rounding Operation Time Values

Rounding Precision	Result	Clock Cycles
Extended	No Underflow, Overflow, or Round Overflow	6
	No Underflow or Overflow; Round Overflow	6
	Underflow	34
	Overflow; RN or RZ Mode; No Round Overflow	14
	Overflow; RM or RP Mode; No Round Overflow	16
	Round Overflow (Not Caused By Rounding); RN or RZ Mode	16
	Round Overflow (Not Caused By Rounding); RM or RP Mode	18
	Round Overflow (Caused By Rounding); RN or RZ Mode	20
	Round Overflow (Caused By Rounding); RM or RP Mode	22
Single or Double	Result is Zero	6
	No Underflow, Overflow, or Round Overflow	24
	No Underflow or Overflow; Round Overflow	28
	Underflow; No Round Overflow	56
	Underflow; Round Overflow	60
	Overflow; RN or RZ Mode; No Round Overflow	30
	Overflow; RM or RP Mode; No Round Overflow	32
	Overflow; RN or RZ Mode; Round Overflow	34
	Overflow; RM or RP Mode; Round Overflow	36

8-18 indicates the number of clock cycles that should be added in the calculation of execution time for an arithmetic instruction (both the total and the overlap allowed numbers) to account for the various rounding precision and exception handling combinations. The entries in the table include the time from the end of the calculation phase to completion of the FPCP instruction execution (i.e., when the PF bit in the null (CA=0) primitive is clear if the response CIR is read).

When an FMOVE instruction that moves data between FP registers is executed in the MC68882 and the FPCR mode control byte specifies single or double precision rounding, no instruction execution concurrency is allowed. Similarly, an FMOVE instruction that moves data to a single or double precision memory location executes without overlap in the MC68882.

Table 8-19 includes the entries referenced previously in the arithmetic calculation and output operand conversion tables for exceptional operand inputs. The values in this table are used for the calculation or conversion timing in lieu of a value from the appropriate table. For example, if an output operand conversion table entry references NAN7, then the timing number from the NAN7 entry in Table 8-19 is used as the conversion time value.

Table 8-19. Exception Handling Time Values

Exception Identifier	Conditions	Clock Cycles
IOP	Source Operand is Not Denormalized	20
	Source Operand is Denormalized	32
NAN1	Destination is a QNAN, Source is not Denormalized	28
	Destination is a QNAN, Source is Denormalized	52
	Destination is an SNAN, Source is not Denormalized	30
	Destination is an SNAN, Source is Denormalized	54
NAN2	The NAN is a QNAN	28
	The NAN is an SNAN	30
NAN3	Both NANs are QNANs	28
	Source is a QNAN, Destination is an SNAN	30
	Source is an SNAN, Destination is a QNAN	32
	Both NANs are SNANs	30
NAN4	The NAN is a QNAN	30
	The NAN is an SNAN	32
NAN5	The NAN is a QNAN	8
	The NAN is an SNAN	10
NAN6	The NAN is a QNAN	38
	The NAN is an SNAN	40
NAN7	The NAN is a QNAN	22
	The NAN is an SNAN	24

8.5.2.7 CONDITIONAL TERMINATION. The effective execution time for the conditional and context switch instructions is not heavily dependent on the FPCP, since the execution of these operations is performed, for the most part, by the MPU. In order to calculate the effective execution time for these instructions, Table 8-20 shows the termination timing for the MPU. The termination processing starts four MPU clock cycles after the end of the response CIR read and ends when the MPU begins execution of the next instruction. Note that the allowed overlap time in this table is always zero, since the FPCP is in the idle state when these instructions reach the termination phase. However, if multiple coprocessors

Table 8-20. Conditional Termination Times Values

Instruction Type		Best Case	Cache Case	Worst Case
FBcc.W	Branch Taken	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	9/0 (2/0/0/0/0)
	Branch Not Taken	4/0 (0/0/0/0/0)	4/0 (0/0/0/0/0)	5/0 (1/0/0/0/0)
FBcc.L	Branch Taken	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	9/0 (2/0/0/0/0)
	Branch Not Taken	4/0 (0/0/0/0/0)	4/0 (0/0/0/0/0)	7/0 (2/0/0/0/0)
FDBcc	True, Not Taken	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	7/0 (1/0/0/0/0)
	False, Not Taken	10/0 (0/0/0/0/0)	10/0 (0/0/0/0/0)	15/0 (3/0/0/0/0)
	False, Taken	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	9/0 (2/0/0/0/0)
FScc	Dn	4/0 (0/0/0/0/0)	4/0 (0/0/0/0/0)	4/0 (1/0/0/0/0)
	(An) + or - (An)*	6/0 (0/0/1/0/0)	8/0 (0/0/1/0/0)	8/0 (1/0/1/0/0)
	Memory**	4/0 (0/0/1/0/0)	6/0 (0/0/1/0/0)	6/0 (1/0/1/0/0)
FTRAPcc	Trap Taken	24/0 (0/1/4/0/0)	25/0 (0/1/4/0/0)	30/0 (2/1/4/0/0)
	Trap Not Taken	4/0 (0/0/0/0/0)	4/0 (0/0/0/0/0)	5/0 (1/0/0/0/0)
FTRAPcc.W	Trap Taken	26/0 (0/1/4/0/0)	27/0 (0/1/4/0/0)	28/0 (2/1/4/0/0)
	Trap Not Taken	6/0 (0/0/0/0/0)	6/0 (0/0/0/0/0)	6/0 (1/0/0/0/0)
FTRAPcc.L	Trap Taken	28/0 (0/1/4/0/0)	29/0 (0/1/4/0/0)	35/0 (3/1/4/0/0)
	Trap Not Taken	8/0 (0/0/0/0/0)	8/0 (0/0/0/0/0)	10/0 (2/0/0/0/0)

*For condition true; subtract one clock for condition false.

**Add the appropriate effective address calculation time.

are used in a system, the execution of other coprocessors may overlap with the execution of these instructions.

In order to determine the execution time for a conditional operation performed by a processor other than an MPU, it is necessary to know the timing for the conditional evaluation by the FPCP. This value is shown in Table 8-20 (in FPCP clock cycles) and indicates the best-case time from the start of the condition CIR write to the end of the response CIR read (which are the only two coprocessor accesses required).

8.5.2.8 MULTIPLE REGISTER TRANSFER. Table 8-21 lists the number of clock cycles and bus cycles required for the MPU to perform a multiple register transfer to or from the FPCP. These transfers occur during the FMOVEM instruction for either the floating-point control register or floating-point data register form of the instruction. The timing values shown in the table include the processing time for either the evaluate effective address and transfer data or transfer multiple coprocessor registers primitive (for the control or data register form, respectively) with CA=1. Assuming that the main processor is an MC68020 or MC68030, the time required to process the null (CA=0) primitive after the transfer is complete must be included.

For the transfer of multiple control registers, the register select list is included in the instruction, and all of the selected registers are transferred as a single operand (from the perspective of the main processor). For the transfer of multiple data registers, the MPU must read the register select mask before starting the register transfer. The amount of time required by the MPU to read and process the register mask is included in the Table 8-21 entries. If a dynamic register list is used, the time required by the MPU to process the transfer single main processor register primitive must be included and is shown at the top of the table.

Table 8-21. Multiple Register Transfer Time Values

Transfer Type		Timing
Move Single Control Register	To an MC68020 Register	17.0 (0'0'0'2'0)
	To Memory*	19.0 (0'0'1'2'0)
	From an MC68020 Register	14.0 (0'0'0'1'1)
	From Memory	19.0 (0'1'0'1'1)
	#(data)	19.0 (1'0'0'1'1)*
Move Multiple Control Registers	To Memory	13 + 6n'0 (0'0'n'1 + n'0)
	From Memory	13 + 6n'0 (0'n'0'1'n)
	#(data)	12 + 6n'0 (n'0'0'1'n)*
Move Multiple Data Registers	To Memory	23 + 25n'0 (0'0'3n'2 + 3n'0)
	From Memory	21 + 23n'0 (0'3n'0'2'3n)

n – is the number of registers transferred.

*If the immediate operand resides in the MPU cache, the number of clock cycles is reduced by 3n and the number of instruction prefetch bus cycles is zero.

8.5.2.9 STATE FRAME TRANSFER. Table 8-22 lists the number of clock cycles and bus cycles required for the MPU to transfer an internal state frame to or from the MC68881. These transfers occur during the FSAVE and FRESTORE instructions. The timing values shown in the table include the time from the end of the instruction start-up operation to the end of the last operand write cycle, assuming that the main processor is an MC68020 or MC68030.

Table 8-22. State Frame Transfer Time Values

Operation	Frame Type	Timing
State Save	Idle	36.0 (0'0'6'6'0)
	Busy	270.0 (0'0'45'45'0)
State Restore	Idle	36.0 (0'6'0'0'6)
	Busy	270.0 (0'45'0'0'45)

Before the transfer of a state frame to the FPCP during an FRESTORE instruction, the MPU must read the format word from memory, write it to the restore CIR, and verify that it is valid by reading the restore CIR. Likewise, during an FSAVE instruction, the MPU must read the format word from the save CIR and store it in memory. The instruction start-up timing table entries include these operations for MC68020/MC68030-based systems.

During an FSAVE operation, the FPCP may require the main processor to wait until the current instruction is completed or a save boundary is reached before starting the state frame transfer. The maximum time that the main processor can be forced to wait is shown at the top of Table 8-22, and should be included in the calculation of the worst-case FSAVE execution time.

In order to calculate overall execution time for the MPU during a FSAVE or FRESTORE instruction, the instruction termination processing time must be included. Table 8-23 lists the timing values for this processing, which is from the end of the last operand write cycle to the beginning of the execution of the next instruction by the MPU.

Table 8-23. Instruction Termination Processing Time Values

Instruction Type	Best Case	Cache Case	Worst Case
FSAVE	1/0 (0'0'0'0'0)	1/0 (0'0'0'0'0)	3'0 (1'0'0'0'0)
FRESTORE	3/0 (0'0'0'0'0)	3/0 (0'0'0'0'0)	4'0 (1'0'0'0'0)

2.10 EXCEPTION PROCESSING. Table 8-24 indicates the time required for exception processing related to the execution of FPCP instructions. The values in the table for the second and third entries indicate the time from the start of processing the take exception primitive until the MPU resumes normal instruction execution in the appropriate exception handler.

Table 8-24. Exception Processing Time Values

Operation	Best Case	Cache Case	Worst Case
Pass Program Counter	2/2* (0/0/0/0/1)	3/3* (0/0/0/0/1)	3/3* (0/0/0/0/1)
Take Pre-Instruction Exception	22/0 (0/1/4/0/0)	22/0 (0/1/4/0/0)	24/0 (2/1/4/0/0)
Take Mid-Instruction Exception	32/0 (0/1/7/0/0)	32/0 (0/1/7/0/0)	38/0 (2/1/7/0/0)
Process Pre-Instruction Interrupt (I stack)	26/26 (0/2/4/0/0)	26/26 (0/2/4/0/0)	33/33 (2/2/4/0/0)
Process Pre-Instruction Interrupt (M stack)	41/41 (0/2/8/0/0)	41/41 (0/2/8/0/0)	48/48 (2/2/8/0/0)
Process Mid-Instruction Interrupt (I stack)	35/35 (0/2/6/0/0)	36/36 (0/2/6/0/0)	42/42 (2/2/6/0/0)
Process Mid-Instruction Interrupt (M stack)	46/46 (0/2/9/0/0)	47/47 (0/2/9/0/0)	53/53 (2/2/9/0/0)
Process FSAVE Interrupt (I stack)	26/26 (0/2/4/0/0)	26/26 (0/2/4/0/0)	33/33 (2/2/4/0/0)
Process FSAVE Interrupt (M stack)	41/41 (0/2/8/0/0)	41/41 (0/2/8/0/0)	48/48 (2/2/8/0/0)
Format Error, FRESTORE Instruction	23/0 (0/1/4/0/0)	24/0 (0/1/4/0/0)	29/0 (2/1/4/0/0)
RTE, Pre-Instruction Frame	20/20 (0/4/0/0/0)	21/21 (0/4/0/0/0)	24/24 (2/4/0/0/0)
RTE, Mid-Instruction Frame	31/24 (0/6/0/1/0)	32/25 (0/6/0/1/0)	33/26 (1/6/0/1/0)
RTE, Throwaway Frame	15/15 (0/4/0/0/0)	16/16 (0/4/0/0/0)	19/19 (0/4/0/0/0)

*Overlap is allowed only for floating-point register-to-register and register-to-external operations.

To determine the overall exception latency for a pre-instruction exception, the instruction start-up time (for the arithmetic or conditional instruction that is pre-empted by the exception) is added to the exception processing time from Table 8-24. The exception processing time for a take mid-instruction exception primitive is added to the overall execution time for the FMOVE to memory instruction that caused the exception. For conditional instructions that cause a BSUN exception, the pass program counter time shown in the table is also added to the instruction start-up and exception processing time to calculate the overall exception latency for the instruction.

For the take interrupt operations, the values in Table 8-24 include the time from the end of the processing of a response primitive that allows interrupts to the resumption of normal MPU instruction execution in the interrupt handler. (The possible responses are the null (CA = 1, IA = 1) and null (CA = 0, IA = 1, PF = 0) primitives, or the not ready format code.) If an interrupt is processed during an FMOVE to memory instruction or when the main processor is in the trace mode and receives a null (CA = 0, IA = 1, PF = 0) primitive, a mid-instruction stack frame is used. A pre-instruction stack frame is used for interrupts processed during an FSAVE instruction. The M-stack and I-stack designation indicates whether the M bit of the MPU status register was set or clear, respectively, before the interrupt occurred.

The processing time for an FRESTORE format error includes the time from the end of the FRESTORE start-up operation to when the MPU resumes normal instruction execution in the format error exception handler. Since the characteristics of an FSAVE format error exception are not predictable (and since such an occurrence is catastrophic), execution timing required to handle the error is not included in the table.

entries in the table for the return from exception (RTE) instruction include the time the beginning of the execution of the RTE by the MPU to the resumption of the previously aborted operation. If the RTE instruction processes a pre-instruction frame, the time in the table includes the time required to restore the processor context and prepare to execute the instruction at the address in the stack frame program counter image. For the mid-instruction frame, the time in the table includes the time required to restore the processor context and read the response CIR to continue the previously suspended operation. The "RTE, throwaway frame" entries include the time required to read and process the throwaway stack frame (normally from the top of the interrupt stack) and then perform RTE processing for the stack frame on top of the resulting active stack (normally either the master or user stack). Thus, if the MPU must return from an interrupt that occurred while the M bit in the MPU status register was set, a throwaway frame is first processed from the interrupt stack, followed by the processing of the appropriate frame from the master stack (which returns the processor to the context saved by the interrupt processing). For such a case, the "RTE, throwaway frame" times are added to the RTE execution times for the second stack frame to derive the overall execution times for the operation.

In addition to the occurrence of an exception, whether exceptions are enabled or not, can also affect instruction execution time. This is because the FPCP requests the transfer of the program counter at the start of any arithmetic instruction if any exception (other than the BSUN exception) is enabled. If the source operand resides in a floating-point data register, the transfer of the PC does not affect overall execution timing, since it takes place concurrently with the execution of the operation by the FPCP. However, for source operands external to the FPCP, the MPU first passes the PC, and then passes the operand; thus, execution time is affected for this case.

8.6 MAIN PROCESSOR INSTRUCTION OVERLAP TIMING

The MPU overlap allowed table for the MC68881 applies to overlap between MPU and floating-point instructions. Table 8-25 lists the overlap time allowed by the MC68881. The MPU overlap time allowed by the MC68882 is shown in the T columns of Table 8-3.

Table 8-25. Overlap Allowed Times — Arithmetic Operations

Operation Type	Fpm Source	Memory Source or Destination Operand Format				
		Integer	Single	Double	Extended	Packed
Fpn Destination*	-3	-22	-22	-28	-34	-34
Move to Dn or Memory**	—	41	29	29	9	113

*Subtract these numbers from the overall execution time value in the previous table to determine the allowed overlap time for a particular instruction.

**These numbers represent the amount of time in the middle of the instruction during which the MPU can process interrupts.

~ Indicates a typical time for the binary-to-decimal conversion.